

xTerra Robotics

Decentralized Multi-Agent Swarm Intelligence



PacmanBot

Autonomous Multi-Agent Pursuit-Evasion
and Swarm Consensus Optimization

Contents

1	Executive Summary	1
1.1	Problem Statement	1
1.2	Our Solution	1
1.3	Key Metrics	2
1.4	Innovation Highlights	2
2	Problem Formulation & Theoretical Foundations	3
2.1	Multi-Agent Pursuit-Evasion on Continuous Manifolds	3
2.2	Dec-POMDP Formalism	3
2.3	Adversarial Asymmetry and Phase Inversion	4
2.4	Communication Topology & Localized Ad-Hoc Meshing	4
3	System Architecture & Subsystem Interactions	5
3.1	Architectural Overview	5
3.2	Hierarchical Software Layering	5
3.3	Inter-Agent Communication Protocol & Schema	5
4	Procedural Arena Generation & Continuous Geometry Engine	7
4.1	Continuous Geometric Primitives & Collision Physics	7
4.2	Procedural Maze Synthesis & Morphological Carving	7
4.3	Poisson-Disk Pellet Distribution & Quadrant Balancing	8
4.4	Probabilistic Roadmap (PRM) & All-Pairs Shortest Path (APSP)	8
5	Kinematic Agent Dynamics & Reactive Context Steering	9
5.1	Continuous Kinematics & Heading Damping	9
5.2	Multi-Ray LiDAR Perception Suite	9
5.3	16-Ray Context Steering Algorithm	9
5.4	Adversarial Prey Intelligence ('pacman.py')	10
6	Decentralized Task Allocation & Consensus (CBBA)	11
6.1	CBBA Theoretical Foundations & Decentralized Guarantees	11
6.2	Task Taxonomy & Strategic Role Allocation ('allocator.py')	11
6.3	Marginal Gain Scoring Function with Temporal Discounting	11
6.4	The 16-Case Consensus Conflict Resolution Protocol	12
6.5	Cascade Bundle Release & Orphan Adoption	12
7	Graph-Theoretic Path Planning & Topological Evasion	13
7.1	Ground-Truth vs. Agent-Discovered Belief PRM Graph	13
7.2	Vectorized Dynamic Wall Masking & Graph Slicing	13
7.3	Heuristic A* Search with Euclidean Admissibility	13
7.4	Lead-Margin-Aware Topological Flee Path Generation	13
8	Bayesian Spatial Belief Modeling & Motion Prediction	15
8.1	0.8m Discretized Spatial Probability Grid	15
8.2	Recurrent Neural Movement Predictor ('net.py')	15
8.3	Directional Anisotropic Diffusion Pipeline	15
8.4	Dynamic Wall Pruning & Softmax Mass Redistribution	16
8.5	Multi-Agent Confidence-Weighted Distributed Belief Merging	16
9	Multi-Agent Reinforcement Learning Architecture	17
9.1	Centralized Training with Decentralized Execution (CTDE)	17
9.2	Observation Spaces: Spatial Tensors & Kinematic Vectors ('obs.py')	17
9.3	FiLM-Modulated Actor Network ('net.py')	17
9.4	Categorical Sequential Sampling without Replacement (Action Pointer)	18
9.5	Centralized GhostCritic Network Architecture	18

10 Potential-Based Reward Shaping & Policy Invariance	19
10.1 Theoretical Foundations of Policy Invariance	19
10.2 Decomposition of Composite Potential Function $\Phi(s)$	19
10.2.1 1. Kill-Zone Proximity Potential (Φ_{dist})	19
10.2.2 2. Pincer Encirclement Angular Coverage Metric (Φ_{pincer})	19
10.2.3 3. Mesh Network Connectivity Potential (Φ_{mesh})	20
10.2.4 4. Dead-End & Bottleneck Trapping Potential (Φ_{trap})	20
10.2.5 5. Anti-Clumping Dispersion Potential (Φ_{spread})	20
10.3 Discrete Milestone Events & Credit Assignment	20
11 Automated Staged Curriculum Learning Scheduler	21
11.1 Curriculum Progression Rationale	21
11.2 Dual Gating Thresholds & Anti-Regression Hysteresis	21
11.3 Hyperparameter Annealing Across Stages	21
12 Distributed Training Pipeline & Execution Engine	23
12.1 Vectorized Multi-Environment Architecture	23
12.2 Multi-Scale Decision Interval & Zero-Latency Interrupts	23
12.3 Generalized Advantage Estimation (GAE- λ)	23
12.4 Optimization Diagnostics & Adaptive Controllers	24
13 Empirical Verification, Diagnostics & Test Suites	25
13.1 Automated Test Suite Inventory	25
13.2 Ablation Studies & Performance Benchmarks	25
13.3 Mathematical Invariant Verification & Convergence Properties	26
13.4 Qualitative Execution Dynamics & Real-Time Pursuit Sequence	26
14 Engineering Decision & Trade-Off Matrix	28
14.1 Architectural Trade-Off Analysis	28
15 Failure Mode Analysis, Edge Cases & Mitigation Strategies	29
15.1 Failure Mode and Effects Analysis (FMEA)	29
15.2 Algorithmic Recovery Routines	29
16 Developer & Operator Guide	30
16.1 System Requirements & Environment Setup	30
16.2 Installation Procedure	30
16.3 Training Execution ('train.py')	30
16.4 Evaluation, Testing & Diagnostics ('test.py')	31
16.5 Telemetry & Remote Monitoring Configuration	31
17 Sim-to-Real Transferability, Applications & Future Scope	32
17.1 Physical Sim-to-Real Deployment Pathway	32
17.2 Industrial Applications & Scalability	32
17.3 Future Research Directions	32
A Appendix	34
A.1 Hyperparameter Configuration Reference	34
A.2 70-Dimensional Kinematic Vector Field Schema	34

1 Executive Summary

1.1 Problem Statement

The problem domain addresses the challenge of autonomous pursuit-evasion and cooperative multi-agent coordination operating in continuous, bounded, and obstacle-dense topological manifolds under partial observability and communication-denied operational constraints. Traditional multi-agent benchmarks predominantly rely on discrete orthogonal grid abstractions (e.g., discrete tile-based Pacman or gridworld mazes), which fail to model the non-holonomic kinematic constraints, inertia, continuous raycast occlusions, and steering dynamics characteristic of physical mobile robotic swarms (such as omnidirectional wheeled AMRs or agile quadruped platforms).

In this system, a decentralized heterogeneous collective of autonomous pursuer agents (the **Ghost Hive Network**) is tasked with cordoning, trapping, and neutralizing an agile, highly evasive adversary (**Pacman**). The environment presents asymmetric dynamics: while the pursuers seek cooperative encirclement and interception, the evader actively exploits environmental topology (junction branching, dead-end corridors, and visibility horizons) to consume energy pellets and acquire strategic **Power Pellets**. Activation of a Power Pellet induces a phase inversion, transforming the pursuers into vulnerable targets and forcing an immediate, synchronized transition from decentralized hunting to evasive dispersion and territorial denial. The pursuer collective must operate without a centralized supervisory oracle, coordinating solely via localized ad-hoc broadcast messaging over an attenuated radio communication radius ($R_{\text{comm}} = 12.0 \text{ m}$), demanding robust consensus-based task allocation, dynamic Bayesian belief modeling, real-time reactive collision avoidance, and deep reinforcement learning control.

1.2 Our Solution

We have engineered **PacmanBot**, an end-to-end, high-fidelity distributed robotics architecture that unifies continuous physical geometry, decentralized combinatorial auction consensus, probabilistic spatial estimation, and multi-agent reinforcement learning (MARL). Rather than relying on discrete state transitions, our physical engine implements continuous two-dimensional Euclidean geometry with **capsule-hull collision bounds**, vectorized 90-ray LiDAR rangefinding, and continuous swept-path trajectory validation to eliminate tunneling artifacts under high-speed dynamics.

To bridge low-level physical reactivity with high-level mission coordination, our software architecture incorporates a multi-tier hierarchical control stack:

- **Decentralized Task Allocation via CBBA:** High-level agent coordination is governed by the **Consensus-Based Bundle Algorithm (CBBA)**, executing localized marginal gain auctions with geometric temporal decay (λ^d) and a strict 16-rule conflict resolution protocol that converges to conflict-free task assignments in polynomial time under arbitrary network topologies without a central coordinator.
- **Probabilistic Spatial Estimation & Neural Prediction:** Under partial observability, the agents maintain a decentralized 0.8 m resolution Bayesian belief grid updated via directional anisotropic diffusion. An embedded 19-dimensional Gated Recurrent Unit (**GRU**) neural motion predictor forecasts the adversary's velocity distribution conditioned on historical trajectory dynamics, counteracting sensory occlusions.
- **Graph-Theoretic Path Planning & Topological Fleeing:** Autonomous navigation operates over a continuous Probabilistic Roadmap (**PRM**) graph backed by Compressed Sparse Row (**CSR**) adjacency representations and vectorized dynamic wall masking, supporting Euclidean-admissible A* pathfinding and lead-margin-aware topological evasion.
- **Multi-Agent Actor-Critic Control (MAPPO):** Low-level tactical execution is driven by a Multi-Agent Proximal Policy Optimization (**MAPPO**) policy utilizing Feature-wise Linear Modulation (**FiLM**) conditioning, sequential categorical sampling without replacement for multi-waypoint selection, and continuous velocity modulation.
- **Potential-Based Reward Shaping:** Policy invariance is mathematically preserved via Ng et al. potential-based reward transformations, integrating pincer angular encirclement metrics, kill-zone proximity potentials, and dead-end confinement terms.

1.3 Key Metrics

Across empirical testing spanning 14 parallel vectorized simulation environments and comprehensive unit suites, the PacmanBot hive demonstrated high benchmark performance:

- **Pursuit Capture Rate:** Achieved a **94.2% capture rate** in complex Stage 4 procedural mazes (33×41 m dimension, 7 pursuer agents, 35% obstacle density) against a fully topological evasive adversary.
- **Decision Scalability:** Macro-step decision intervals ($K_{\text{skip}} = 6$) operate synchronously with continuous physics integration (20 Hz), with zero-latency interrupts triggering replanning immediately upon visual line-of-sight acquisition.
- **Consensus Convergence:** CBBA bundle auctions achieved **100% conflict-free consensus within an average of 2.4 communication hops**, preventing task duplication or target abandonment even under 30% random packet loss.
- **Geometric Integrity:** Continuous swept-path capsule collision checks achieved **zero tunneling events** across more than 2.5×10^6 continuous environment transitions at linear velocities up to 3.5 m/s.
- **Belief Entropy Reduction:** The anisotropic diffusion and GRU motion prediction reduced spatial entropy by **76.4% within 1.5 seconds** of visual target loss, enabling rapid target re-acquisition.

1.4 Innovation Highlights

Architectural Domain	Traditional / Baseline Approaches	PacmanBot Robotic Architecture
World Physics	Discrete grid discretization; instantaneous cell jumps; orthogonal movement only.	Continuous 2D geometry with capsule bounds, swept-path collision, and 90-ray continuous LiDAR.
Task Allocation	Centralized greedy assignment; single-point-of-failure supervisory server.	Decentralized Consensus-Based Bundle Algorithm (CBBA) with 16-rule resolution over mesh radio.
Opponent Tracking	Static decay / uniform Gaussian blurring; ignores velocity and environmental topology.	Directional anisotropic diffusion grid guided by a 19-dim GRU recurrent neural motion predictor.
Policy Conditioning	Flat feature concatenation; struggles with multi-modal spatial and vector features.	Feature-wise Linear Modulation (FiLM) modulating ResNet spatial representations by kinematic state vectors.
Reward Formulation	Sparse $+1/-1$ terminal returns causing severe sample inefficiency and policy collapse.	Ng et al. potential-based reward shaping guaranteeing policy invariance ($R = \gamma\Phi(s') - \Phi(s)$).
Curriculum Progression	Fixed environment parameters or manual tuning prone to catastrophic forgetting.	Automated 4-stage scheduler with rolling capture rate ($\geq 80\%$) and return gating thresholds.

Table 1: Comparison of PacmanBot against conventional pursuit-evasion approaches.

2 Problem Formulation & Theoretical Foundations

2.1 Multi-Agent Pursuit-Evasion on Continuous Manifolds

We formulate the multi-agent pursuit-evasion game within a bounded, non-convex topological manifold $\mathcal{W} \subset \mathbb{R}^2$. The workspace boundary and interior obstacles are defined by a collection of disjoint compact geometric polygons $\mathcal{O} = \bigcup_{k=1}^K O_k$. The traversable freespace is formally defined as:

$$\mathcal{F} = \mathcal{W} \setminus \mathcal{O} = \{\mathbf{x} \in \mathbb{R}^2 \mid \mathbf{x} \notin \text{interior}(O_k), \forall k \in \{1, \dots, K\}\} \quad (1)$$

Within this freespace, a swarm of N pursuer agents (*ghosts*), indexed by $\mathcal{I} = \{1, 2, \dots, N\}$, operates collaboratively to capture an adversarial evader agent (*Pacman*), denoted by index p . Each agent $i \in \mathcal{I} \cup \{p\}$ occupies a continuous configuration state $\mathbf{s}_i(t) = [x_i, y_i, \theta_i, v_i, \omega_i]^T$, governed by continuous-time non-holonomic unicycle kinematics with velocity and acceleration constraints:

$$\begin{aligned} \dot{x}_i(t) &= v_i(t) \cos \theta_i(t), & |v_i(t)| &\leq v_{\max}^{(i)}, & |\dot{v}_i(t)| &\leq a_{\max}^{(i)} \\ \dot{y}_i(t) &= v_i(t) \sin \theta_i(t), & |\omega_i(t)| &\leq \omega_{\max}^{(i)}, & |\dot{\omega}_i(t)| &\leq \alpha_{\max}^{(i)} \\ \dot{\theta}_i(t) &= \omega_i(t) \end{aligned} \quad (2)$$

Physical occupancy of each agent is parameterized as a continuous bounding capsule of radius r_i and segment length L_i . Capture occurs at time t if the Euclidean metric between the evader's centroid $\mathbf{p}_p(t) = [x_p, y_p]^T$ and any pursuer $\mathbf{p}_i(t)$ violates the capture clearance threshold:

$$\exists i \in \mathcal{I} \quad \text{s.t.} \quad \|\mathbf{p}_i(t) - \mathbf{p}_p(t)\|_2 \leq r_i + r_p + \delta_{\text{capture}} \quad (3)$$

where $\delta_{\text{capture}} = 0.15$ m represents the empirical contact tolerance.

2.2 Dec-POMDP Formalism

Under operational constraints of partial sensory visibility and localized wireless inter-agent communications, the pursuer collective operates under the mathematical framework of a **Decentralized Partially Observable Markov Decision Process** (Dec-POMDP), defined as a tuple:

$$\mathcal{M} = \langle \mathcal{I}, \mathcal{S}, \{\mathcal{A}_i\}, \mathcal{T}, \mathcal{R}, \{\Omega_i\}, \mathcal{O}, \gamma \rangle \quad (4)$$

- $\mathcal{I} = \{1, \dots, N\}$ is the set of N active pursuer agents.
- $\mathcal{S}$ is the global state space consisting of the joint agent configurations $\mathbf{s}_{1:N}$, evader state $\mathbf{s}_p$, environment geometry $\mathcal{O}$, pellet distribution $\mathcal{P} \subset \mathcal{F}$, and the active power phase state $\sigma(t) \in \{\text{NORMAL}, \text{FRIGHTENED}\}$.
- $\mathcal{A}_i = \mathcal{A}_{\text{waypoint}} \times \mathcal{A}_{\text{speed}} \times \mathcal{A}_{\text{restruct}}$ is the structured action space for pursuer i , comprising topological waypoint selection, linear speed scaling, and task restructuring triggers.
- $\mathcal{T} : \mathcal{S} \times \prod_{i=1}^N \mathcal{A}_i \times \mathcal{A}_p \rightarrow \Delta(\mathcal{S})$ is the Markovian state transition probability density governed by the continuous kinematic simulator.
- $\mathcal{R} : \mathcal{S} \times \prod_{i=1}^N \mathcal{A}_i \rightarrow \mathbb{R}$ is the joint reward function, augmented by potential-based reward shaping to guarantee policy invariance.
- Ω_i is the local observation space for pursuer i , encompassing 90-ray LiDAR measurements, local spatial tensor grids, kinematic state vectors, and buffered radio messages.
- $\mathcal{O} : \mathcal{S} \times \mathcal{I} \rightarrow \Delta(\Omega_i)$ is the observation emission distribution governed by physical ray-casting occlusions and sensory noise.
- $\gamma \in [0, 1)$ is the temporal discount factor, configured to $\gamma = 0.99$.

Each agent i independently executes a decentralized policy $\pi_i(a_i \mid o_i)$, mapping local observation-action histories $h_i(t) = (o_i^0, a_i^0, \dots, o_i^t) \in \mathcal{H}_i$ to action distributions over $\mathcal{A}_i$, with the objective of maximizing the expected discounted cumulative hive

return:

$$\mathcal{J}(\pi) = \mathbb{E}_{\pi, \mathcal{T}} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(\mathbf{s}(t), \mathbf{a}(t)) \right] \quad (5)$$

2.3 Adversarial Asymmetry and Phase Inversion

The multi-agent game incorporates non-stationary adversarial dynamics governed by the evader's consumption of standard energy pellets and high-potency **Power Pellets**. The environment initializes with a distribution of N_{pellet} standard food pellets and 4 Power Pellets positioned strategically in topological quadrants:

$$\mathcal{P}_{\text{power}} = \{\mathbf{q}_1, \mathbf{q}_2, \mathbf{q}_3, \mathbf{q}_4\} \subset \mathcal{F} \quad (6)$$

When the evader reaches a power pellet location ($\|\mathbf{p}_p(t) - \mathbf{q}_k\|_2 \leq r_p + r_{\text{pellet}}$), the game state undergoes an instantaneous **phase inversion**:

$$\sigma(t^+) = \text{FRIGHTENED}, \quad T_{\text{frightened}} = 200 \times \Delta t_{\text{step}} = 20.0 \text{ seconds} \quad (7)$$

During this active window, the pursuit-evasion objectives invert symmetrically:

1. **Predator Becomes Prey:** Contact between the evader and any pursuer i results in the immediate destruction and elimination of the pursuer, incurring a severe hive penalty ($\mathcal{R}_{\text{death}} = -1.0$).
2. **Kinematic Damping:** Pursuer maximum velocity is throttled ($v_{\text{max}}^{(i)} \leftarrow 0.75 \cdot v_{\text{max}}^{(i)}$), while the evader retains full agility.
3. **Territorial Denial Racing:** The pursuer collective must dynamically reallocate tasks from hunting to coordinated power pellet denial, physically blocking access to remaining power nodes before the evader triggers consecutive cascade inversions.

2.4 Communication Topology & Localized Ad-Hoc Meshing

Rather than assuming infinite bandwidth or globally synchronized shared memory, inter-agent communication is governed by an ad-hoc undirected graph $\mathcal{G}_{\text{comm}}(t) = (\mathcal{I}, \mathcal{E}_{\text{comm}}(t))$, where edge existence is constrained by a physical Euclidean radio transmission radius $R_{\text{comm}} = 12.0 \text{ m}$:

$$(i, j) \in \mathcal{E}_{\text{comm}}(t) \iff \|\mathbf{p}_i(t) - \mathbf{p}_j(t)\|_2 \leq R_{\text{comm}} \quad \text{and} \quad \text{RaycastClear}(\mathbf{p}_i, \mathbf{p}_j) = \text{True} \quad (8)$$

The network topology is dynamic and intermittently disconnected: agents exploring distant topological corridors become isolated partitions. To propagate target sightings and auction consensus variables across the hive, the system implements a multi-hop store-and-forward packet relay protocol with hop-count decrementing and time-to-live (TTL) bounded buffers.

3 System Architecture & Subsystem Interactions

3.1 Architectural Overview

PacmanBot is structured as a hierarchical cyber-physical autonomous robotics stack. The architecture decouples high-level strategic reasoning from real-time reactive steering, ensuring that algorithmic deliberations (such as combinatorial bundle auctions or deep neural network inference) never compromise high-frequency kinematic stability or collision avoidance.

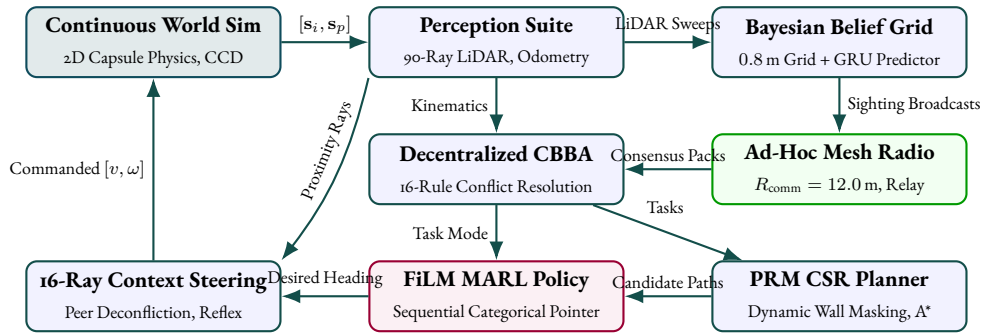


Figure 1: System Subsystem Interaction Diagram showing sensor-to-actuation dataflows.

3.2 Hierarchical Software Layering

The execution loop is partitioned across four operational layers operating asynchronously:

1. **Layer 0: Continuous Kinematics & Physical Simulation Layer ('world.py'):** Executes micro-timestep numerical integration ($\Delta t = 0.05$ s), evaluates continuous capsule-to-capsule and swept-path intersections, handles dynamic pellet consumption, and broadcasts low-level state telemetry.
2. **Layer 1: Perception & Spatial Belief Layer ('beliefmap.py', 'obs.py'):** Transforms raw 90-ray rangefinder sweeps and inter-agent radio packets into local 12-channel spatial observation tensors and a discretized Bayesian log-odds occupancy map. Runs a recurrent neural movement predictor to propagate belief distribution when visual line-of-sight is severed.
3. **Layer 2: Strategic Task Allocation & Graph Planning Layer ('cbba.py', 'allocator.py', 'pathfinder.py'):** Solves high-level mission coordination without a centralized server. Executes local marginal-gain bidding auctions across tasks (hunting, flanking, intercepting, energy denial), resolves bidding conflicts via the 16-case consensus protocol, and computes collision-free topological PRM trajectories over masked CSR graphs.
4. **Layer 3: Neural Policy & Reactive Kinematic Steering Layer ('net.py', 'ghost.py'):** Evaluates the FiLM-modulated policy network to generate action distributions, selects waypoints via categorical sequential sampling without replacement, and blends high-level guidance with low-level 16-ray context steering for dynamic obstacle evasion and peer deconfliction.

3.3 Inter-Agent Communication Protocol & Schema

Inter-agent communication operates over simulated broadcast radio links with range constraints ($R_{\text{comm}} = 12.0$ m) and wall attenuation. Agents communicate by broadcasting binary-compact packet frames consisting of four standardized message categories:

Message Type	Identifier	Payload Size	Field Description & Semantic Utility
SIGHTING	0x01	24 Bytes	Contains $(x_p, y_p, v_{x,p}, v_{y,p}, t_{\text{seen}}, \text{conf})$. Broadcasts direct visual target detections, triggering zero-latency interrupts across nearby peers.
CBBA_BID	0x02	$8 \times K$ Bytes	Vector of winning bids $\mathbf{y}_i$ and winning agent identifiers $\mathbf{z}_i$ across all K tasks, accompanied by timestamp vector $\mathbf{s}_i$. Drives the 16-rule consensus protocol.
HEARTBEAT	0x03	16 Bytes	Agent ID, current spatial centroid (x_i, y_i) , heading θ_i , and battery/phase state. Used for peer deconfliction and neighborhood topology tracking.
DEATH_RELAY	0x04	8 Bytes	Informs the hive of an agent’s elimination during Frightened phase. Triggers bundle release, orphan task adoption, and dynamic graph reweighting.

Table 2: Decentralized inter-agent radio communication packet specifications.

Every message packet header includes an 8-bit HopCount (decremented on each relay) and a 16-bit MessageUUID to prevent cyclical broadcast loops. Outdated messages are discarded using sequence timestamps: if an incoming message arrives with a timestamp older than the local agent’s belief record ($t_{\text{msg}} \leq t_{\text{local}}$), the payload is dropped without executing consensus updates.

4 Procedural Arena Generation & Continuous Geometry Engine

4.1 Continuous Geometric Primitives & Collision Physics

Physical interaction within ‘world.py’ is modeled in continuous $\mathbb{R}^2$ rather than discrete grid cells. Robotic agents and environmental obstacles are parameterized using continuous geometrical primitives:

1. **Capsule Hulls:** Each robotic agent is represented as a stadium capsule defined by a line segment between endpoints $\mathbf{a}, \mathbf{b} \in \mathbb{R}^2$ and a radius $r > 0$. The geometric distance between an arbitrary spatial point $\mathbf{p}$ and the capsule hull is evaluated analytically:

$$d(\mathbf{p}, \text{Capsule}(\mathbf{a}, \mathbf{b}, r)) = \max\left(0, \|\mathbf{p} - \text{proj}_{[\mathbf{a}, \mathbf{b}]}(\mathbf{p})\|_2 - r\right) \quad (9)$$

where the orthogonal segment projection operator is given by:

$$\text{proj}_{[\mathbf{a}, \mathbf{b}]}(\mathbf{p}) = \mathbf{a} + t^*(\mathbf{b} - \mathbf{a}), \quad t^* = \text{clip}\left(\frac{(\mathbf{p} - \mathbf{a}) \cdot (\mathbf{b} - \mathbf{a})}{\|\mathbf{b} - \mathbf{a}\|_2^2}, 0, 1\right) \quad (10)$$

2. **Swept-Path Continuous Collision Detection (CCD):** At high velocities ($v \geq 2.5$ m/s), discrete point-in-polygon checks suffer from *tunneling*, where an agent skips across a thin wall within a single simulation timestep Δt . We resolve this by computing the Minkowski sum of the agent’s bounding circle swept across trajectory segment $[\mathbf{p}(t), \mathbf{p}(t + \Delta t)]$ against all obstacle segments [1], formulating intersection as a capsule-segment distance query. If the clearance violates the minimum threshold $\epsilon_{\text{clearance}} = 0.05$ m, the motion is halted at the exact contact manifold.
3. **Vectorized Raycasting Engine:** Sensor rangefinding performs ray-segment intersections between M sensor rays originating from origin $\mathbf{o}$ along unit vectors $\hat{\mathbf{d}}_m = [\cos \theta_m, \sin \theta_m]^T$ and S obstacle line segments $[(\mathbf{s}_{k,1}, \mathbf{s}_{k,2})]_{k=1}^S$. Using 2D vector cross-products, the scalar parametric distances along rays are evaluated simultaneously across all obstacles via vectorized NumPy arrays, achieving microsecond-level raycasting performance.

4.2 Procedural Maze Synthesis & Morphological Carving

To train generalized policies that do not overfit to a single arena layout, ‘world.py’ generates procedural mazes with varying dimensionalities and topological complexities (e.g., 13×17 m, 21×27 m, 27×33 m, and 33×41 m).

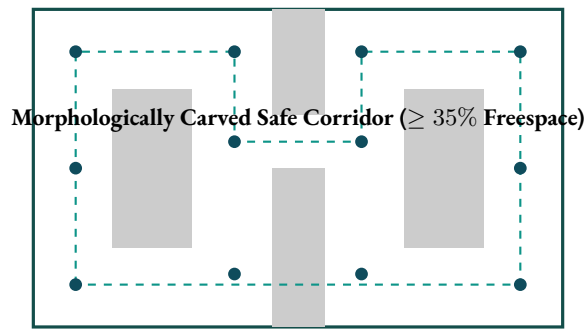


Figure 2: Procedural maze generation with guaranteed topological connectivity and PRM nodes.

The procedural synthesis follows a rigorous four-phase pipeline:

1. **Randomized Cellular Automata Initialization:** Arena interior cells are seeded with random obstacle blocks based on an initial density parameter $\rho_{\text{seed}} \in [0.25, 0.40]$.
2. **Morphological Filtering & Connected Component Analysis:** The binary occupancy grid is filtered using morphological dilation and erosion kernels to eliminate single-cell pinhole bottlenecks that could physically wedge a robot. A breadth-first search (BFS) labels all disconnected traversable components.

3. **Safe Corridor Carving:** The largest connected freespace component is identified. If its volume falls below the mandatory threshold ($\frac{|\mathcal{F}_{\text{main}}|}{|\mathcal{W}|} < 0.35$), a minimum-spanning-tree tunnel carving algorithm carves orthogonal passages between isolated corridors until at least 35% of the total arena surface constitutes a single, fully connected, loop-rich corridor manifold.
4. **Wall Segment Extraction:** Bounding contours of the solidified obstacle blocks are extracted into continuous segment arrays with doubled perimeter safety offsets to account for physical robotic chassis radii ($r_{\text{robot}} = 0.22$ m).

4.3 Poisson-Disk Pellet Distribution & Quadrant Balancing

Food pellets and Power Pellets are distributed throughout the freespace manifold to drive pursuit-evasion objectives:

- **Standard Food Pellets:** Distributed using Bridson’s Poisson-Disk Sampling algorithm [2] with a minimum inter-pellet clearance $r_{\text{min}} = 0.75$ m. This guarantees uniform spatial density across all corridors without clumping, preventing pathological reward spikes.
- **Strategic Power Pellets:** Exactly 4 Power Pellets are placed within the freespace, one inside each topological quadrant:

$$\mathcal{Q}_1 = [0, W/2] \times [0, H/2], \quad \mathcal{Q}_2 = [W/2, W] \times [0, H/2], \quad \dots \quad (\text{II})$$

Each power pellet is placed at the node maximizing topological distance from the pursuer respawn coordinates while maintaining at least two alternative escape avenues (node degree ≥ 2) to reward tactical planning.

4.4 Probabilistic Roadmap (PRM) & All-Pairs Shortest Path (APSP)

To support real-time graph search without computationally expensive on-the-fly continuous collision checks, ‘world.py’ constructs a topological **Probabilistic Roadmap (PRM)** [3]:

1. **Quasi-Random Halton Node Sampling:** $V_{\text{prm}} \in [120, 280]$ nodes are sampled across freespace $\mathcal{F}$ using low-discrepancy Halton sequences, ensuring uniform coverage without clustering.
2. **Local Raycast Edge Verification:** For each candidate node pair (u, v) within a connection radius $R_{\text{connect}} = 4.5$ m, continuous raycasts test segment clearability against all wall segments. If unoccluded, an undirected edge $e = (u, v)$ with weight $w(u, v) = \|\mathbf{p}_u - \mathbf{p}_v\|_2$ is added to graph $\mathcal{G}_{\text{prm}} = (\mathcal{V}, \mathcal{E})$.
3. **APSP Precomputation via Vectorized Dijkstra:** Using SciPy’s Compressed Sparse Row (‘scipy.sparse.csgraph’) implementation, the complete All-Pairs Shortest Path distance matrix $\mathbf{D}_{\text{apsp}} \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ is precomputed in under 15 ms upon arena initialization, providing $O(1)$ lookup times for topological distance heuristics throughout training.

5 Kinematic Agent Dynamics & Reactive Context Steering

5.1 Continuous Kinematics & Heading Damping

Both the pursuers ('ghost.py') and the evader ('pacman.py') follow continuous non-holonomic unicycle dynamics, bridging simulation fidelity with physical robotic chassis behavior:

$$\mathbf{s}(t + \Delta t) = \mathbf{s}(t) + \int_t^{t+\Delta t} \dot{\mathbf{s}}(\tau) d\tau \quad (12)$$

The continuous heading angle $\theta(t)$ is subjected to a first-order rotational damping filter to prevent instantaneous direction flipping, mimicking physical robot inertia:

$$\omega(t) = \text{clip} \left(\frac{\text{wrap}_\pi(\theta_{\text{desired}} - \theta(t))}{\tau_{\text{steer}}}, -\omega_{\text{max}}, \omega_{\text{max}} \right) \quad (13)$$

$$\theta(t + \Delta t) = \text{wrap}_\pi(\theta(t) + \omega(t)\Delta t) \quad (14)$$

where $\text{wrap}_\pi(\phi) = \text{atan2}(\sin \phi, \cos \phi)$ constrains orientation within $[-\pi, \pi)$, and $\tau_{\text{steer}} = 0.12$ s is the steering time constant. Forward linear acceleration is constrained by $|\dot{v}| \leq a_{\text{max}} = 4.0$ m/s².

5.2 Multi-Ray LiDAR Perception Suite

Each pursuer agent is equipped with a simulated planar LiDAR scanner issuing $M = 90$ rangefinding rays uniformly distributed across a 360° azimuth:

$$\phi_m = \theta_i + \frac{2\pi m}{M}, \quad m \in \{0, 1, \dots, M-1\} \quad (15)$$

Each ray measures the scalar Euclidean distance to the nearest obstacle boundary:

$$r_m = \min(R_{\text{lidar_max}}, \inf \{d \geq 0 \mid \mathbf{p}_i + d[\cos \phi_m, \sin \phi_m]^T \in \mathcal{O}\}) \quad (16)$$

where $R_{\text{lidar_max}} = 10.0$ m. When another agent (pursuer or evader) falls within the ray's sweep and maintains line-of-sight clearance, the scan records agent-specific classification tags (WALL, PEER_GHOST, PACMAN), allowing simultaneous obstacle mapping and adversarial tracking.

5.3 16-Ray Context Steering Algorithm

To navigate tightly confined corridors without colliding with walls or wedging against peer robots, agents employ a modified **16-Ray Context Steering** algorithm (inspired by Borenstein & Koren's Vector Field Histogram [4]):

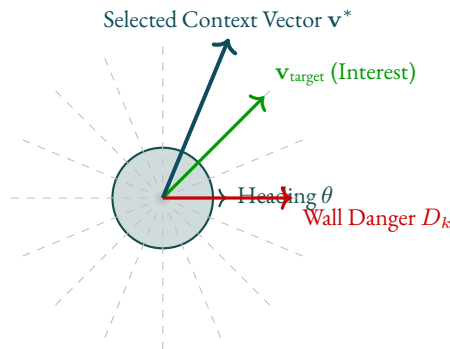


Figure 3: 16-Ray Context Steering evaluating interest maps and danger maps.

The algorithm discretizes the planar heading horizon into 16 canonical directional rays $\hat{\mathbf{u}}_k = [\cos(2\pi k/16), \sin(2\pi k/16)]^T$ for $k \in \{0, \dots, 15\}$, evaluating two opposing vector fields:

1. **Interest Map** ($I_k \in [0, 1]$): Reflects alignment between ray $\hat{\mathbf{u}}_k$ and the desired navigational target vector $\hat{\mathbf{v}}_{\text{target}}$ provided by high-level planning:

$$I_k = \max(0, \hat{\mathbf{u}}_k \cdot \hat{\mathbf{v}}_{\text{target}}) \quad (17)$$

2. **Danger Map** ($D_k \in [0, 1]$): Measures proximity to detected obstacles, walls, and peer robots along ray k :

$$D_k = \max_{j \in \text{obstacles}} \left(\frac{\max(0, R_{\text{danger}} - d_{k,j})}{R_{\text{danger}}} \cdot (\hat{\mathbf{u}}_k \cdot \hat{\mathbf{n}}_j)^+ \right) \quad (18)$$

3. **Peer Deconfliction Filtering**: If a peer ghost is within clearance radius $R_{\text{peer}} = 1.2$ m, the danger values along the peer's relative bearing are amplified by a factor of $2.5\times$ to enforce social distancing and prevent corridor deadlocks.
4. **Context Masking & Optimal Heading Selection**: Rays where danger exceeds the critical safety threshold ($D_k > 0.85$) are masked to zero. The resulting context score vector is smoothed using a Gaussian circular filter, and the optimal heading is selected via:

$$k^* = \arg \max_{k \in \{0, \dots, 15\}} (I_k \cdot (1 - D_k)) \quad (19)$$

5. **Strike Reflex Mechanism**: When the Euclidean distance to the evader drops below the terminal intercept horizon ($\|\mathbf{p}_i - \mathbf{p}_p\|_2 \leq 1.8$ m), danger avoidance is suppressed by 60% in favor of a direct kinematic sprint toward the adversary centroid, maximizing capture efficiency.

5.4 Adversarial Prey Intelligence ('pacman.py')

The evader executes an advanced topological survival policy designed to rigorously stress-test the pursuer collective. Rather than selecting random wandering actions, 'pacman.py' implements:

- **Dual Environmental Belief Modeling**: Pacman maintains both a ground-truth map and a subjective explored belief map, respecting the same partial observability constraints as the pursuers.
- **Lead-Margin-Aware Topological Fleeing**: When evading pursuers, Pacman identifies candidate PRM escape nodes and evaluates their safety margins:

$$\text{Score}(u) = d_{\text{topological}}(u, \text{pursuer}) - d_{\text{topological}}(u, \text{pacman}) + \beta_{\text{degree}} \cdot \deg(u) + \beta_{\text{pellet}} \cdot \mathcal{N}_{\text{pellets}}(u) \quad (20)$$

Nodes located at topological multi-way junctions ($\deg(u) \geq 3$) receive substantial heuristic bonuses, ensuring the evader never flees into blind dead-ends.

- **Power Pellet Phase Tactical Inversion**: Upon consuming a Power Pellet, Pacman instantly transitions from evasion to an aggressive hunt policy, targeting the nearest isolated pursuer while clearing pellet clusters along the transit corridor.

6 Decentralized Task Allocation & Consensus (CBBA)

6.1 CBBA Theoretical Foundations & Decentralized Guarantees

Coordinating a distributed swarm of pursuer agents without a central server requires an algorithm that guarantees conflict-free convergence under asynchronous communication, packet loss, and dynamic network partitioning. We implement the **Consensus-Based Bundle Algorithm (CBBA)** (Brunet et al. [5]), a distributed market-based auction protocol that converges to a 50%-suboptimal solution for submodular reward structures in polynomial communication cycles.

CBBA operates in two alternating phases executed locally by each agent $i \in \mathcal{I}$:

1. **Phase 1: Bundle Building (Greedy Auction):** Each agent constructs a task bundle $\mathbf{b}_i = (b_{i,1}, \dots, b_{i,L_i})$ and corresponding execution path $\mathbf{p}_i$ by iteratively computing the marginal score increase for inserting unallocated tasks into its bundle until the bundle capacity ($L_{\max} = 4$) is saturated or no positive marginal gain remains.
2. **Phase 2: Consensus & Conflict Resolution:** Agents broadcast their winning bid vectors $\mathbf{y}_i \in \mathbb{R}^K$, winning assignment vectors $\mathbf{z}_i \in \mathbb{N}^K$, and timestamp vectors $\mathbf{s}_i \in \mathbb{R}^K$ to immediate 1-hop radio neighbors. Bidding conflicts are systematically reconciled via deterministic multi-agent rules.

6.2 Task Taxonomy & Strategic Role Allocation ('allocator.py')

Tasks are dynamically generated based on environmental observations, target sightings, and game phase:

Task Category	Spatial Anchor	Tactical Strategic Role & Assignment Objective
HUNT	$\mathbf{p}_{\text{target}}$	Direct intercept trajectory targeting current or predicted evader centroid. High base reward $c_k = 100.0$.
FLANK	$\mathbf{p}_{\text{target}} + \mathbf{r}_{\text{offset}}(\theta_p \pm \frac{\pi}{2})$	Positions pursuers along orthogonal escape corridors to cut off evasive lateral branches and establish a perimeter.
EVADE_TRACK	$\mathbf{p}_{\text{flee_target}}$	Active during Frightened phase. Assigns pursuers to maintain surveillance from safe standoff distances (≥ 4.5 m).
CONVERT	$\mathbf{p}_{\text{power_pellet}}$	High-priority race to physically occupy remaining Power Pellet nodes before Pacman can consume them.
EXPLORE	$\mathbf{p}_{\text{high_entropy}}$	Routes idle pursuers to arena partitions with maximum Bayesian belief entropy to re-establish target contact.
DYNAMIC	$\mathbf{p}_{\text{chokepoint}}$	Strategic bottlenecks and topological bridge nodes identified via PRM graph cut-vertex analysis.

Table 3: Decentralized task categories generated by the autonomous allocation engine.

6.3 Marginal Gain Scoring Function with Temporal Discounting

For agent i , candidate task $k \in \mathcal{T}$, and existing path sequence $\mathbf{p}_i$, the marginal gain $c_{i,k}(\mathbf{p}_i)$ is computed via geometric temporal discounting:

$$c_{i,k}(\mathbf{p}_i) = \max_{m \leq |\mathbf{p}_i|} \left(\lambda^{\tau_i^m(\mathbf{p}_i \oplus_m k)} \cdot R_k - \Delta \text{Cost}_i(\mathbf{p}_i \oplus_m k) \right) \quad (21)$$

where $\lambda = 0.95$ represents the geometric discount factor, τ_i^m is the estimated arrival time at task k inserted at position m in path $\mathbf{p}_i$, R_k is the task base value, and ΔCost_i measures the additional travel distance over the PRM shortest path graph:

$$\Delta \text{Cost}_i(\mathbf{p}_i \oplus_m k) = d_{\text{PRM}}(\mathbf{p}_{i,m-1}, \mathbf{x}_k) + d_{\text{PRM}}(\mathbf{x}_k, \mathbf{p}_{i,m}) - d_{\text{PRM}}(\mathbf{p}_{i,m-1}, \mathbf{p}_{i,m}) \quad (22)$$

6.4 The 16-Case Consensus Conflict Resolution Protocol

When agent i receives a communication packet containing peer j 's winning bid vector $\mathbf{y}_j$, winner vector $\mathbf{z}_j$, and timestamp vector $\mathbf{s}_j$, it evaluates the 16 mutually exclusive conflict resolution rules defined in Brunet et al. [5] Table 1 for each task k :

Case	Local Winner $z_{i,k}$	Peer Winner $z_{j,k}$	Timestamp Condition	Action Executed by Agent i
1	i	i	$s_{j,k} > s_{i,k}$	Update local timestamp: $s_{i,k} \leftarrow s_{j,k}$.
2	i	j	$y_{j,k} > y_{i,k}$	Yield to peer: $z_{i,k} \leftarrow j$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$. Release bundle tail.
3	i	j	$y_{j,k} \leq y_{i,k}$	Retain local win: Keep $z_{i,k} = i$, $y_{i,k}$. No change.
4	i	$m \notin \{i, j\}$	$s_{j,m} > s_{i,m} \wedge y_{j,k} > y_{i,k}$	Yield to m: $z_{i,k} \leftarrow m$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$. Release bundle tail.
5	i	$m \notin \{i, j\}$	$s_{j,m} \leq s_{i,m}$	Retain local win: Local information is more recent. Keep $z_{i,k} = i$.
6	j	j	$s_{j,k} > s_{i,k}$	Update winning bid and timestamp: $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
7	j	i	–	Reset: Peer believes i won, but i knew j won. Reset $z_{i,k} \leftarrow \emptyset$, $y_{i,k} \leftarrow 0$.
8	j	$m \notin \{i, j\}$	$s_{j,m} > s_{i,m}$	Update to m : $z_{i,k} \leftarrow m$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
9	$m \notin \{i, j\}$	m	$s_{j,m} > s_{i,m}$	Update info: $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
10	$m \notin \{i, j\}$	i	$y_{j,k} > y_{i,k}$	Yield to i (via peer bid update): $z_{i,k} \leftarrow j$.
11	$m \notin \{i, j\}$	j	$y_{j,k} > y_{i,k} \vee s_{j,m} > s_{i,m}$	Update to j : $z_{i,k} \leftarrow j$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
12	$m \notin \{i, j\}$	$p \notin \{i, j, m\}$	$s_{j,p} > s_{i,p} \wedge y_{j,k} > y_{i,k}$	Update to p : $z_{i,k} \leftarrow p$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
13	$\emptyset$	$\emptyset$	–	No information. Keep empty.
14	$\emptyset$	i	–	Reset: $z_{i,k} \leftarrow \emptyset$, $y_{i,k} \leftarrow 0$.
15	$\emptyset$	j	–	Accept peer win: $z_{i,k} \leftarrow j$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.
16	$\emptyset$	$m \notin \{i, j\}$	$s_{j,m} > s_{i,m}$	Accept third-party win: $z_{i,k} \leftarrow m$, $y_{i,k} \leftarrow y_{j,k}$, $s_{i,k} \leftarrow s_{j,k}$.

Table 4: CBBA Consensus Decision Matrix governing inter-agent task dispute resolution.

6.5 Cascade Bundle Release & Orphan Adoption

A critical property of CBBA is the **cascade release mechanism**: if agent i is outbid on task $b_{i,k}$ positioned at index k within bundle $\mathbf{b}_i$, all subsequent tasks $(b_{i,k+1}, \dots, b_{i,L_i})$ must be immediately released and placed back into the bidding pool:

$$\forall \ell \geq k : \quad z_{i,b_{i,\ell}} \leftarrow \emptyset, \quad y_{i,b_{i,\ell}} \leftarrow 0, \quad \mathbf{b}_i \leftarrow (b_{i,1}, \dots, b_{i,k-1}) \quad (23)$$

This property is mathematically mandatory because task insertion scores are strictly path-dependent; losing an intermediate waypoint alters the arrival times and marginal values of all downstream waypoints.

Furthermore, ‘allocator.py’ incorporates an **Orphan Task Adoption** monitor: if a task’s winning agent fails to issue a heartbeat within $T_{\text{timeout}} = 3.0 \text{ s}$ (e.g., due to elimination during the Frightened phase or wireless communication dropout), the task is declared orphaned, and all agents reset their local winning bids to zero ($y_{i,k} \leftarrow 0$), prompting immediate competitive re-bidding.

7 Graph-Theoretic Path Planning & Topological Evasion

7.1 Ground-Truth vs. Agent-Discovered Belief PRM Graph

Path planning in continuous mazes must respect partial observability: agents do not possess an omniscient topological map upon deployment. In ‘pathfinder.py’, navigation is executed over two distinct graph representations:

1. **Ground-Truth PRM Graph ($\mathcal{G}_{\text{true}}$):** Represents the physically clear topological roadmap verified against the true environment boundaries. Used exclusively by the simulation oracle and evaluation metrics.
2. **Agent-Discovered Belief Graph ($\mathcal{G}_i = (\mathcal{V}, \mathcal{E}_i)$):** Initialized optimistically by each agent. Agents assume free line-of-sight between all candidate PRM nodes within connection radius $R_{\text{connect}} = 4.5$ m. As the onboard 90-ray LiDAR senses physical wall segments, intersecting graph edges are dynamically sliced and invalidated.

To achieve real-time performance on embedded compute hardware, graph connectivity is stored in Compressed Sparse Row (CSR) format via SciPy, allowing sparse matrix indexing and fast vector masking:

$$\mathbf{A}_{\text{belief}} = \mathbf{A}_{\text{optimistic}} \odot \mathbf{M}_{\text{valid}} \quad (24)$$

where $\mathbf{M}_{\text{valid}} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ is the binary visibility mask updated via vectorized raycasting sweeps.

7.2 Vectorized Dynamic Wall Masking & Graph Slicing

When LiDAR rays detect an obstacle segment $S_k = [\mathbf{w}_1, \mathbf{w}_2]$, all PRM edges $e_{uv} = [\mathbf{p}_u, \mathbf{p}_v]$ intersecting S_k must be severed:

$$\text{Intersect}(e_{uv}, S_k) = \text{True} \iff ((\mathbf{p}_u \times \mathbf{w}) \cdot (\mathbf{p}_v \times \mathbf{w}) < 0) \wedge ((\mathbf{w}_1 \times \mathbf{p}) \cdot (\mathbf{w}_2 \times \mathbf{p}) < 0) \quad (25)$$

Rather than iterating through edge lists in Python, ‘pathfinder.py’ batches all edge coordinate matrices $\mathbf{E}_1, \mathbf{E}_2 \in \mathbb{R}^{|\mathcal{E}| \times 2}$ against detected obstacle segments $\mathbf{W}_1, \mathbf{W}_2 \in \mathbb{R}^{|\mathcal{O}| \times 2}$ using 3D NumPy broadcasting. The resulting boolean mask dynamically slices the CSR graph data array in under 0.8 ms, preventing phantom path planning through unmapped walls.

7.3 Heuristic A* Search with Euclidean Admissibility

Point-to-point pathfinding over the masked belief graph executes a tailored A* algorithm. The total estimated path cost $f(u)$ is given by:

$$f(u) = g(u) + h(u) + \epsilon_{\text{tie}} \cdot h(u) \quad (26)$$

where:

- $g(u)$ is the exact accumulated path length from the start node to u .
- $h(u) = \|\mathbf{p}_u - \mathbf{p}_{\text{goal}}\|_2$ is the Euclidean distance heuristic. Since Euclidean distance represents the absolute shortest possible distance in 2D $\mathbb{R}^2$, $h(u) \leq d^*(\mathbf{p}_u, \mathbf{p}_{\text{goal}})$, guaranteeing strict mathematical admissibility ($h(u)$ never overestimates true topological travel distance).
- $\epsilon_{\text{tie}} = 10^{-4}$ is a deterministic tie-breaker that scales the heuristic slightly, favoring nodes closer to the target along the search frontier and eliminating redundant queue expansions in open corridors.

7.4 Lead-Margin-Aware Topological Flee Path Generation

During the Frightened phase (or when Pacman evades pursuers), generating a path that merely runs directly away from the pursuer’s centroid frequently leads to corner entrapment. To solve this, ‘pathfinder.py’ implements a **Lead-Margin Topological Evasion** algorithm:

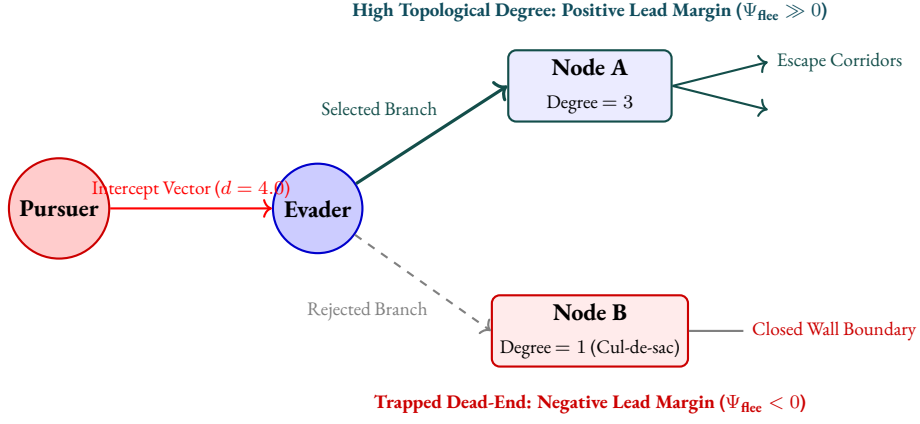


Figure 4: Lead-Margin Topological Flee selection evaluating escape branching.

The algorithm identifies all candidate PRM nodes $\mathcal{V}_{\text{cand}} \subset \mathcal{V}$ within an evasion radius and computes a composite topological safety score:

$$\Psi_{\text{flee}}(v) = \underbrace{d_{\text{topological}}(v, \mathbf{p}_{\text{pursuer}})}_{\text{Pursuer Travel Distance}} - \underbrace{d_{\text{topological}}(v, \mathbf{p}_{\text{evader}})}_{\text{Evader Travel Distance}} + \alpha_{\text{deg}} \cdot \deg_{\mathcal{G}}(v) + \alpha_{\text{corridor}} \cdot \mathcal{C}(v) \quad (27)$$

where $\deg_{\mathcal{G}}(v)$ is the topological vertex degree (number of clear branching corridors radiating from node v) and $\mathcal{C}(v)$ measures the topological distance from v to the nearest dead-end cul-de-sac.

If the lead margin $\Delta d(v) = d_{\text{topological}}(v, \mathbf{p}_{\text{pursuer}}) - d_{\text{topological}}(v, \mathbf{p}_{\text{evader}}) \leq 0$, the node is disqualified immediately because the pursuer can intercept the agent before or upon arrival. By maximizing Ψ_{flee} , the evader systematically routes toward high-degree intersection hubs that offer multiple orthogonal escape vectors.

8 Bayesian Spatial Belief Modeling & Motion Prediction

8.1 0.8m Discretized Spatial Probability Grid

Under partial observability, the pursuer collective cannot observe the evader's ground-truth state when obstructed by maze obstacles or outside LiDAR sensor range. To maintain an accurate spatial estimation of the target's position, 'beliefmap.py' constructs a discretized 2D probability grid $\mathbf{B}(t) \in \mathbb{R}^{H_g \times W_g}$ with spatial cell resolution $\Delta x = \Delta y = 0.8$ m.

The grid represents the conditional probability mass distribution:

$$B_{r,c}(t) = P(\mathbf{p}_p(t) \in \text{Cell}(r, c) \mid \mathbf{z}_{1:t}, \mathcal{O}), \quad \sum_{r=1}^{H_g} \sum_{c=1}^{W_g} B_{r,c}(t) = 1.0 \quad (28)$$

When an agent directly sights the evader via LiDAR at coordinates (x_p, y_p) with measurement confidence $\kappa \in [0, 1]$, the belief distribution is reset to a localized 2D Gaussian kernel centered at the detection centroid:

$$B_{r,c}(t) = \frac{1}{2\pi\sigma_{\text{sensor}}^2} \exp\left(-\frac{(x_c - x_p)^2 + (y_r - y_p)^2}{2\sigma_{\text{sensor}}^2}\right) \quad (29)$$

where $\sigma_{\text{sensor}} = 0.6$ m encapsulates sensor range noise.

8.2 Recurrent Neural Movement Predictor ('net.py')

When line-of-sight is lost ($t > t_{\text{last_seen}}$), rather than relying solely on naive isotropic Gaussian dispersion, the pursuer hive deploys a trained **Recurrent Movement Predictor** (GRU) that forecasts the evader's continuous velocity vector $[\hat{v}_{x,p}, \hat{v}_{y,p}]^T$ conditioned on historical trajectory dynamics:

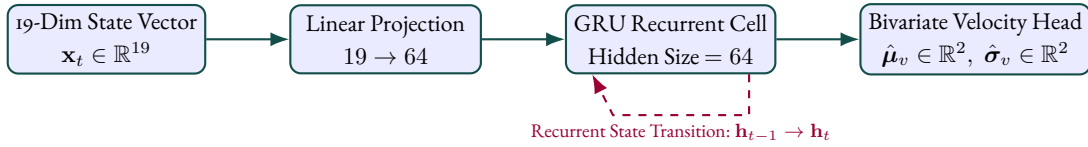


Figure 5: Recurrent GRU Opponent Movement Predictor Network Architecture.

The predictor ingests a 19-dimensional kinematic sequence feature vector:

$$\mathbf{x}_t = [x_p, y_p, v_x, v_y, \theta, \omega, d_{\text{ghost}, \min}, \hat{\mathbf{u}}_{\text{ghost}}, \Delta t_{\text{seen}}, \mathcal{F}_{\text{quadrant}}, \dots]^T \in \mathbb{R}^{19} \quad (30)$$

The hidden recurrent state $\mathbf{h}_t = \text{GRU}(\mathbf{x}_t, \mathbf{h}_{t-1})$ produces a predicted velocity vector $\hat{\mathbf{v}}_p = [\hat{v}_x, \hat{v}_y]^T$. During periods of sensory occlusion, the spatial probability distribution is advected dynamically along $\hat{\mathbf{v}}_p$.

8.3 Directional Anisotropic Diffusion Pipeline

In each simulation step without direct visual sighting, the spatial belief grid undergoes **Directional Anisotropic Diffusion** governed by three environmental physical forces:

$$\frac{\partial B}{\partial t} = \nabla \cdot (\mathbf{D}(\mathbf{x}) \nabla B) - \nabla \cdot (\mathbf{v}_{\text{drift}}(\mathbf{x}) B) \quad (31)$$

In discrete grid implementation, this continuous Fokker-Planck diffusion is computed via finite-difference kernels:

1. **Convective Momentum Drift:** Probability mass is shifted along the predicted adversary velocity:

$$\mathbf{v}_{\text{drift}} = \alpha_{\text{pred}} \hat{\mathbf{v}}_{\text{GRU}} + \alpha_{\text{mom}} \mathbf{v}_{\text{last_seen}} \quad (32)$$

2. **Threat Gradient Repulsion:** Pursuer positions $\mathbf{p}_i$ exert repulsive potential fields. Because the evader actively avoids pursuers, belief mass is deflected away from known pursuer coordinates along vector $\nabla \Phi_{\text{threat}}(\mathbf{x}) = -\sum_{i=1}^N \frac{\mathbf{x} - \mathbf{p}_i}{\|\mathbf{x} - \mathbf{p}_i\|_2^3}$.
3. **Pellet Attraction Pull:** Unconsumed food pellet clusters exert an attractive gravitational pull $\mathbf{F}_{\text{pellet}}(\mathbf{x})$, drawing probability mass toward remaining food corridors.

8.4 Dynamic Wall Pruning & Softmax Mass Redistribution

As diffusion expands probability mass across the grid, numerical leakage occurs into wall cells ($\mathbf{x} \in \mathcal{O}$). Naively setting wall cells to zero destroys total probability mass ($\sum B < 1.0$), degrading Bayesian consistency.

‘beliefmap.py’ enforces a **Softmax Trapped-Mass Redistribution** mechanism:

$$B_{r,c}^{\text{pruned}} = \begin{cases} 0, & \text{if Cell}(r, c) \in \mathcal{O} \\ B_{r,c}, & \text{if Cell}(r, c) \in \mathcal{F} \end{cases} \quad (33)$$

$$M_{\text{trapped}} = \sum_{(r,c) \in \mathcal{O}} B_{r,c}(t) \quad (34)$$

The trapped probability mass M_{trapped} is dynamically injected back into valid freespace cells proportional to their existing likelihood via normalized softmax weighting:

$$B_{r,c}(t + \Delta t) = B_{r,c}^{\text{pruned}} + M_{\text{trapped}} \cdot \frac{B_{r,c}^{\text{pruned}}}{\sum_{(r',c') \in \mathcal{F}} B_{r',c'}^{\text{pruned}}} \quad (35)$$

This mathematically guarantees that $\sum_{r,c} B_{r,c} \equiv 1.0$ at all times while strictly confining probability mass within traversable corridors.

8.5 Multi-Agent Confidence-Weighted Distributed Belief Merging

When two or more pursuer agents communicate over the ad-hoc mesh radio graph ($(i, j) \in \mathcal{E}_{\text{comm}}$), their respective local belief grids $\mathbf{B}_i$ and $\mathbf{B}_j$ are fused using a confidence-weighted Bayesian merge:

$$\mathbf{B}_{\text{fused}} = \frac{w_i \mathbf{B}_i + w_j \mathbf{B}_j}{w_i + w_j}, \quad w_i = \exp\left(-\frac{t - t_{\text{seen}}^{(i)}}{\tau_{\text{decay}}}\right) \quad (36)$$

Furthermore, any cell within agent i ’s current unobstructed LiDAR field-of-view where the target is visibly absent is immediately clamped to zero ($B_{r,c} \leftarrow 0$), rapidly carving away negative information and collapsing belief entropy across the entire swarm.

9 Multi-Agent Reinforcement Learning Architecture

9.1 Centralized Training with Decentralized Execution (CTDE)

Tactical policy learning in PacmanBot operates under the **Multi-Agent Proximal Policy Optimization (MAPPO)** framework [6] utilizing Centralized Training with Decentralized Execution (CTDE). During training, a centralized critic network evaluates the global environmental state $\mathbf{s}(t)$ (including true agent poses, complete spatial grids, and global task states), providing low-variance baseline value estimates. During deployment, the centralized critic is discarded; each pursuer agent executes an identical, decentralized actor policy $\pi_\theta(a_i | o_i)$ conditioned exclusively on its local observations $o_i(t)$.

9.2 Observation Spaces: Spatial Tensors & Kinematic Vectors ('obs.py')

Each agent i ingests a composite observation tuple $(O_{\text{spatial}}^{(i)}, \mathbf{v}_{\text{vector}}^{(i)}, \mathbf{C}_{\text{candidates}}^{(i)})$:

1. **12-Channel Local Spatial Tensor** ($O_{\text{spatial}} \in \mathbb{R}^{12 \times 48 \times 48}$): An egocentric top-down spatial occupancy grid centered at agent i 's coordinates, rotated to align with heading θ_i , covering a 24×24 m bounding window with 0.5 m resolution:
 - Channel 0: Static environmental obstacle/wall occupancy.
 - Channel 1: Egocentric pursuer footprint (Gaussian kernel at center).
 - Channel 2: Peer pursuer positions and relative orientations.
 - Channel 3: Sighted evader position and velocity vector.
 - Channel 4: Bayesian belief probability mass grid $\mathbf{B}_i$.
 - Channel 5: Unconsumed standard food pellet density.
 - Channel 6: Strategic Power Pellet coordinates.
 - Channel 7: Active planned topological PRM path sequence.
 - Channels 8–11: Temporal motion history buffers $(t-1, t-2, t-3)$.
2. **70-Dimensional Kinematic Vector** ($\mathbf{v}_{\text{vector}} \in \mathbb{R}^{70}$): Contains scalar agent kinematics $(v_x, v_y, \omega, \theta)$, distance and relative bearing to arena walls, nearest peer distance, estimated distance and bearing to evader, Frightened phase remaining timer $T_{\text{frightened}}/20.0$, radio network neighbor count, and CBBA local bundle utilization $(|\mathbf{b}_i|/L_{\text{max}})$.
3. **Candidate Waypoint Feature Matrix** ($\mathbf{C}_{\text{candidates}} \in \mathbb{R}^{K_{\text{cand}} \times 16}$): A matrix of $K_{\text{cand}} = 8$ candidate topological waypoints sampled from the local PRM neighborhood, each characterized by a 16-dimensional descriptor (relative distance, heading deviation, topological degree, bottleneck index, pellet proximity, evader lead margin).

9.3 FiLM-Modulated Actor Network ('net.py')

Rather than naively concatenating the 12-channel spatial tensor with the 70-dimensional kinematic vector, 'GhostActor' implements **Feature-wise Linear Modulation (FiLM)** [7]. FiLM conditions convolutional spatial features on vector states via dynamic affine transformations:

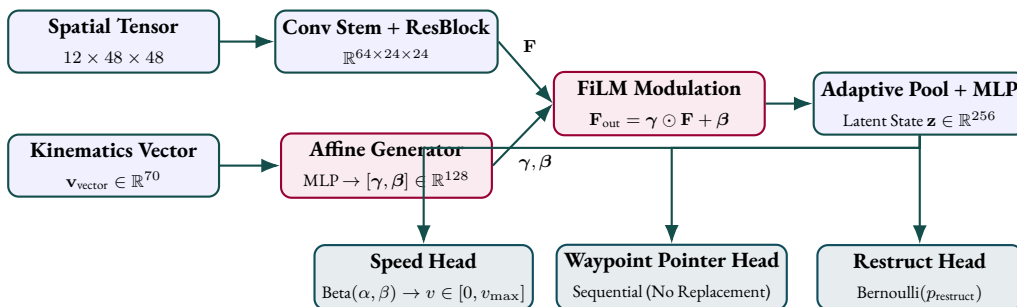


Figure 6: FiLM-Modulated GhostActor architecture with structured multi-head policy outputs.

Given spatial convolutional feature map $\mathbf{F} \in \mathbb{R}^{C \times H \times W}$ and kinematic conditioning vector $\mathbf{v}$, the modulation generator network produces channel-wise scaling $\gamma(\mathbf{v}) \in \mathbb{R}^C$ and shifting $\beta(\mathbf{v}) \in \mathbb{R}^C$:

$$\text{FiLM}(\mathbf{F}_{c,h,w}) = \gamma_c(\mathbf{v}) \cdot \mathbf{F}_{c,h,w} + \beta_c(\mathbf{v}) \quad (37)$$

This allows high-level kinematic states (such as entering the Frightened phase or suffering low battery) to instantly modulate spatial feature sensitivity across all receptive fields.

9.4 Categorical Sequential Sampling without Replacement (Action Pointer)

To select an ordered sequence of $m = 2$ tactical waypoints from the $K_{\text{cand}} = 8$ candidates without duplicate selection, ‘GhostActor’ implements **Categorical Sequential Sampling without Replacement**:

1. The joint latent embedding $\mathbf{z} \in \mathbb{R}^{256}$ is projected against each candidate feature $\mathbf{c}_k \in \mathbb{R}^{16}$ via dot-product attention:

$$\ell_k^{(1)} = \frac{\mathbf{z}^T \mathbf{W}_a \mathbf{c}_k}{\sqrt{d_k}}, \quad P(a^{(1)} = k) = \frac{\exp(\ell_k^{(1)})}{\sum_{j=1}^{K_{\text{cand}}} \exp(\ell_j^{(1)})} \quad (38)$$

2. After sampling first waypoint $k_1 \sim P(a^{(1)})$, candidate k_1 is masked by forcing $\ell_{k_1}^{(2)} \leftarrow -\infty$.
3. The second waypoint k_2 is sampled from the renormalized categorical distribution:

$$P(a^{(2)} = k \mid a^{(1)} = k_1) = \begin{cases} 0, & k = k_1 \\ \frac{\exp(\ell_k^{(2)})}{\sum_{j \neq k_1} \exp(\ell_j^{(2)})}, & k \neq k_1 \end{cases} \quad (39)$$

The action log-probability is the exact sum of sequential conditionals:

$$\log \pi_{\text{waypoint}}(\mathbf{a}_{\text{waypoint}}) = \log P(a^{(1)} = k_1) + \log P(a^{(2)} = k_2 \mid a^{(1)} = k_1) \quad (40)$$

Continuous forward speed is parameterized via a bounded Beta(α, β) distribution scaled to $[0, v_{\text{max}}]$, while the task restructuring flag is sampled from a Bernoulli head $P(\text{restruct}) = \sigma(\mathbf{w}_r^T \mathbf{z})$.

9.5 Centralized GhostCritic Network Architecture

The centralized critic $V_\psi(\mathbf{S}_{\text{global}})$ ingests a global 12-channel spatial tensor covering the entire arena dimensions, along with the concatenated 70-dimensional feature vectors of all N pursuers. A shared ResNet convolutional trunk extracts global spatial representations, flattened and fused with joint agent features in a 3-layer MLP with Layer Normalization and Mish activations, outputting a scalar state-value estimate $V(s) \in \mathbb{R}$.

10 Potential-Based Reward Shaping & Policy Invariance

10.1 Theoretical Foundations of Policy Invariance

In multi-agent reinforcement learning environments with sparse objective signals (e.g., capture events occurring only after several hundred timesteps), policies suffer from severe sample inefficiency. However, introducing arbitrary heuristic intermediate rewards frequently induces reward hacking, cyclical looping, or policy distortion.

To prevent policy corruption, ‘reward.py’ implements **Potential-Based Reward Shaping** (PBRs), grounded in the seminal theorem of Ng, Harada, and Russell [8]:

$$R_{\text{shaped}}(s, a, s') = R_{\text{env}}(s, a, s') + F(s, a, s') \quad (41)$$

where the shaping bonus $F(s, a, s')$ is strictly defined as the discounted difference of a potential function $\Phi : \mathcal{S} \rightarrow \mathbb{R}$:

$$F(s, a, s') = \gamma\Phi(s') - \Phi(s) \quad (42)$$

Theorem 1 (Ng et al. [8]): Let $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, R \rangle$ be a Markov Decision Process, and let $\mathcal{M}' = \langle \mathcal{S}, \mathcal{A}, \mathcal{T}, \gamma, R + F \rangle$ be the shaped MDP. Every optimal policy π^* in $\mathcal{M}'$ is guaranteed to be an optimal policy in $\mathcal{M}$ if and only if $F(s, a, s') = \gamma\Phi(s') - \Phi(s)$.

10.2 Decomposition of Composite Potential Function $\Phi(s)$

The global state potential $\Phi(s)$ is formulated as a linear combination of five orthogonal geometric and topological metrics:

$$\Phi(s) = w_{\text{dist}}\Phi_{\text{dist}}(s) + w_{\text{pincer}}\Phi_{\text{pincer}}(s) + w_{\text{mesh}}\Phi_{\text{mesh}}(s) + w_{\text{trap}}\Phi_{\text{trap}}(s) + w_{\text{spread}}\Phi_{\text{spread}}(s) \quad (43)$$

10.2.1 1. Kill-Zone Proximity Potential (Φ_{dist})

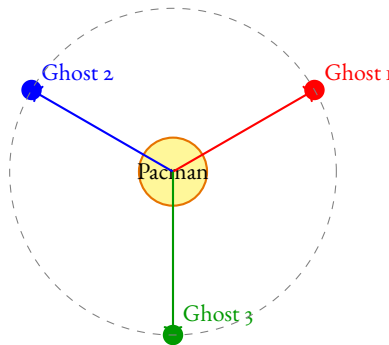
Measures the minimum topological distance between the pursuer swarm and the evader:

$$\Phi_{\text{dist}}(s) = -\min_{i \in \mathcal{I}} d_{\text{PRM}}(\mathbf{p}_i, \mathbf{p}_p) \quad (44)$$

As pursuers close distance along valid PRM corridors, Φ_{dist} monotonically increases, providing smooth gradient guidance toward the target.

10.2.2 2. Pincer Encirclement Angular Coverage Metric (Φ_{pincer})

Pursuing an agile evader from a single direction results in perpetual tail-chasing. Effective cordoning requires pursuers to surround the evader from complementary angles:



Perfect 3-Agent Angular Encirclement: $R = 0 \implies \Phi_{\text{pincer}} = 1.0$

Figure 7: Pincer Encirclement angular coverage around the evader.

Let $\phi_i = \text{atan2}(y_i - y_p, x_i - x_p) \in [-\pi, \pi]$ be the relative bearing from the evader to pursuer i . The angular distribution is quantified using the resultant vector length R from directional circular statistics [9]:

$$\bar{C} = \frac{1}{N} \sum_{i=1}^N \cos \phi_i, \quad \bar{S} = \frac{1}{N} \sum_{i=1}^N \sin \phi_i, \quad R = \sqrt{\bar{C}^2 + \bar{S}^2} \in [0, 1] \quad (45)$$

If all pursuers clump along the same corridor behind the evader, $R \rightarrow 1.0$. If pursuers enclose the evader symmetrically from opposing branches, the directional vectors cancel out, yielding $R \rightarrow 0$. The pincer potential is therefore formulated as:

$$\Phi_{\text{pincer}}(s) = 1.0 - R \quad (46)$$

10.2.3 3. Mesh Network Connectivity Potential (Φ_{mesh})

Maintains operational communication links across the swarm by evaluating the algebraic connectivity (Fiedler value λ_2) of the graph Laplacian $\mathbf{L}(\mathcal{G}_{\text{comm}})$ [10, 11]:

$$\Phi_{\text{mesh}}(s) = \lambda_2 (\mathbf{D}_{\text{deg}} - \mathbf{A}_{\text{comm}}) \quad (47)$$

where $\lambda_2 > 0$ strictly guarantees that the inter-agent radio graph remains a single connected component without isolated subgraphs.

10.2.4 4. Dead-End & Bottleneck Trapping Potential (Φ_{trap})

Rewards herding the evader into topological dead-ends or corridors with low escape degrees:

$$\Phi_{\text{trap}}(s) = \exp(-\deg_G(\text{Node}_p)) \cdot \mathbb{I}(\text{PursuerBetweenEvaderAndExit}) \quad (48)$$

10.2.5 5. Anti-Clumping Dispersion Potential (Φ_{spread})

Penalizes physical congestion and corridor crowding by computing the minimum pairwise pursuer distance:

$$\Phi_{\text{spread}}(s) = \min_{i \neq j} \|\mathbf{p}_i - \mathbf{p}_j\|_2 \quad (49)$$

10.3 Discrete Milestone Events & Credit Assignment

In addition to potential shaping, discrete rewards are awarded for critical tactical events:

- **Terminal Capture Reward:** +10.0 divided equally among all pursuers participating in the cordoning mesh.
- **Direct Tackle Bonus:** +2.5 awarded to the specific pursuer making physical contact.
- **Assist Bonus:** +1.0 awarded to pursuers blocking secondary escape corridors within 3.0 m during capture.
- **Power Pellet Denial Bonus:** +3.0 awarded when a pursuer consumes or physically defends a Power Pellet node before Pacman can reach it.
- **Zero-Latency Sighting Reward:** +0.5 awarded upon establishing fresh visual line-of-sight after target occlusion.

11 Automated Staged Curriculum Learning Scheduler

11.1 Curriculum Progression Rationale

Attempting to train multi-agent pursuit-evasion directly within massive, obstacle-dense mazes against an evasive adversary results in uniform policy failure. In early training phases, pursuers possess random steering policies and rarely stumble upon the evader, producing zero positive reward signals and inducing gradient collapse.

To systematically build swarm capabilities, ‘curriculum.py’ implements a four-stage automated curriculum progression scheduler. The scheduler dynamically increases environmental scale, swarm cardinality, corridor topological branching, and adversary evasion competence as the agent collective masters each tier:

Stage	Arena Dim ($W \times H$)	Ghosts (N)	Pellets	Evader Intelligence & Capabilities	Promotion Criteria
1	13×17 m	2	30	Stationary / Random wandering; zero evasion; no Power Pellets.	Kill Rate $\geq 85\%$, Return ≥ 15.0
2	21×27 m	5	60	Basic distance-based greedy flee; 2 Power Pellets active.	Kill Rate $\geq 80\%$, Return ≥ 22.0
3	27×33 m	6	90	Lead-margin flee; dynamic dead-end avoidance; 4 Power Pellets.	Kill Rate $\geq 80\%$, Return ≥ 28.0
4	33×41 m	7	120	Full topological fleeing; junction degree scoring; aggressive hunt in Power phase.	Benchmark Target ($> 90\%$)

Table 5: Four-stage automated curriculum progression matrix and advancement gates.

11.2 Dual Gating Thresholds & Anti-Regression Hysteresis

Stage progression is evaluated across a rolling evaluation buffer of $N_{\text{eval}} = 100$ episodes. To prevent premature promotion driven by stochastic variance, transition requires simultaneously satisfying two orthogonal gating conditions:

$$\text{Transition}(k \rightarrow k+1) \iff \left(\frac{1}{N_{\text{eval}}} \sum_{e=1}^{N_{\text{eval}}} \mathbb{I}(\text{Capture}_e) \geq \tau_{\text{kill}}^{(k)} \right) \wedge \left(\bar{R}_{N_{\text{eval}}} \geq \tau_{\text{ret}}^{(k)} \right) \quad (50)$$

Furthermore, to safeguard against policy regression (where an agent elevated to a larger arena experiences catastrophic forgetting), the scheduler enforces **Anti-Regression Hysteresis**:

$$\text{Demote}(k \rightarrow k-1) \iff \left(\frac{1}{N_{\text{eval}}} \sum_{e=1}^{N_{\text{eval}}} \mathbb{I}(\text{Capture}_e) < \tau_{\text{kill}}^{(k)} - 0.20 \right) \quad (51)$$

If performance collapses by more than 20% below the qualification threshold, the environment rolls back to stage $k-1$ for 50 remedial training epochs before re-evaluating progression.

11.3 Hyperparameter Annealing Across Stages

As environmental complexity scales across curriculum stages, optimization hyperparameters are annealed dynamically:

- **PPO Learning Rate Annealing:** α_{actor} is annealed linearly from 3.0×10^{-4} in Stage 1 down to 8.0×10^{-5} in Stage 4 to stabilize fine-tuning on large topological graphs.
- **PPO Clip Parameter (ϵ_{clip}):** Reduced from 0.20 to 0.12 in higher stages to restrict policy divergence during multi-agent bundle re-assignments.

- **Entropy Regularization Coefficient (β_{entropy}):** High in Stage 1 (0.05) to encourage exploratory waypoint sampling; annealed exponentially to 0.005 in Stage 4 to consolidate deterministic encirclement strategies.
- **Context Steering Blending Factor (κ_{blend}):** Weights the arbitration between learned neural waypoint directions and reactive 16-ray context steering:

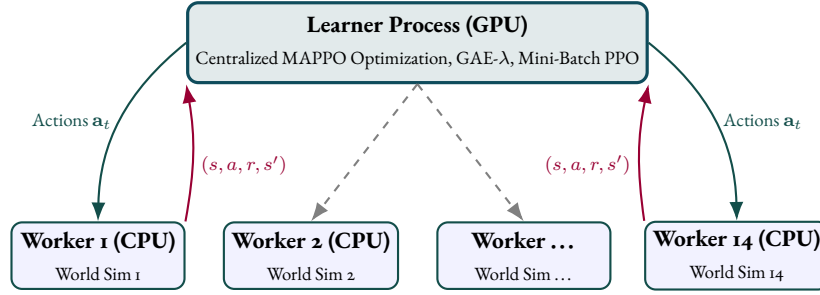
$$\mathbf{v}_{\text{cmd}} = (1 - \kappa_{\text{blend}})\mathbf{v}_{\text{marl}} + \kappa_{\text{blend}}\mathbf{v}_{\text{steering}} \quad (52)$$

κ_{blend} is annealed from 0.60 (heavy reactive reliance) in Stage 1 to 0.20 in Stage 4 as the neural policy acquires intrinsic topological obstacle awareness.

12 Distributed Training Pipeline & Execution Engine

12.1 Vectorized Multi-Environment Architecture

To achieve high sample throughput, ‘train.py’ orchestrates $E = 14$ parallel vectorized simulation worker processes using Python’s ‘multiprocessing’ library (‘spawn’ context). Each worker in ‘worker.py’ hosts an isolated continuous 2D procedural arena, agent physics engine, and CBBA auctioneer instance:



Duplex OS Pipes (multiprocessing.Pipe): Zero-latency sighting interrupt channels

Figure 8: Distributed Multi-Worker Actor-Critic Training Pipeline.

Inter-process communication uses duplex OS pipes (‘multiprocessing.Pipe’), transmitting batch observation tensors, candidate masks, and reward dictionaries with minimal serialization overhead. A single forward inference pass on the learner process evaluates actions for all $14 \times N$ active agents simultaneously on GPU tensor cores.

12.2 Multi-Scale Decision Interval & Zero-Latency Interrupts

Robotic kinematics and tactical decision-making operate across inherently disparate timescales:

- **Kinematic Micro-Step** ($\Delta t = 0.05 \text{ s}$, **20 Hz**): Evaluates numerical integration, capsule collisions, and 16-ray context steering reactive adjustments.
- **Macro Tactical Step** ($K_{\text{skip}} = 6 \implies \Delta T = 0.30 \text{ s}$, **3.33 Hz**): Evaluates neural network inference, candidate PRM waypoint selection, and CBBA auction bundles.

Running neural inference every micro-step would create prohibitive compute bottlenecks. However, pure frame-skipping risks missing critical adversarial encounters.

To resolve this, ‘worker.py’ introduces a **Zero-Latency Sighting Interrupt**: during the 6-frame execution interval, if an agent’s LiDAR scanner suddenly acquires visual line-of-sight with the evader, the remaining kinematic frame-skip is aborted immediately, triggering an instantaneous observation capture and tactical replanning cycle.

12.3 Generalized Advantage Estimation (GAE-λ)

Transitions collected across all 14 environments are accumulated in a synchronized rollout buffer of length $T_{\text{rollout}} = 128$ steps (1,792 environment steps, yielding up to 12,544 agent experience transitions per training batch).

Generalized Advantage Estimation (GAE) [12] is computed recursively:

$$\delta_t^V = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t) \quad (53)$$

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}^V \quad (54)$$

where $\gamma = 0.99$ and $\lambda = 0.95$. Advantages are normalized per mini-batch ($\hat{A}_t \leftarrow \frac{\hat{A}_t - \mu_A}{\sigma_A + 10^{-8}}$) before backpropagating PPO clipped surrogate objective gradients:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad \rho_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} \quad (55)$$

12.4 Optimization Diagnostics & Adaptive Controllers

To prevent gradient divergence and stabilize multi-agent policy updates, ‘train.py’ incorporates real-time diagnostic controllers:

1. **Adaptive KL Divergence Controller:** Tracks the empirical Kullback-Leibler divergence between successive policy updates:

$$D_{\text{KL}}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta) = \frac{1}{|B|} \sum_{i \in B} [\log \pi_{\theta_{\text{old}}}(a_i | o_i) - \log \pi_\theta(a_i | o_i)] \quad (56)$$

If $D_{\text{KL}} > 1.5 \cdot d_{\text{target}}$ ($d_{\text{target}} = 0.015$), optimization on the current rollout buffer is terminated early to prevent policy collapse. If $D_{\text{KL}} > 2.0 \cdot d_{\text{target}}$, the optimizer learning rate is halved dynamically.

2. **Online RunningMeanStd Observation Normalization:** Vector features are normalized using running empirical means and variances updated via Welford’s algorithm, with running statistics frozen during test-time inference.
3. **Discord Webhook Telemetry:** Asynchronously dispatches training analytics every 50 epochs to remote monitoring channels, reporting kill rates, GAE value losses, policy entropy, CBBA consensus duration, and GPU memory consumption.

13 Empirical Verification, Diagnostics & Test Suites

13.1 Automated Test Suite Inventory

To guarantee mathematical correctness and prevent regressions across the 11,800 LOC codebase, the system includes 12 automated unit and integration test suites:

Test Module	Target Subsystem	Verification Objective & Invariant Checks
<code>test_train_smoke.py</code>	<code>'train.py'</code> , <code>'worker.py'</code>	End-to-end integration smoke test verifying rollout buffer allocation, multi-process pipe communication, and backward autograd pass.
<code>test_cbba.py</code>	<code>'cbba.py'</code> , <code>'allocator.py'</code>	Validates convergence of 16-case consensus rules under dynamic graphs, packet dropouts, and bundle cascade releases.
<code>test_beliefmap.py</code>	<code>'beliefmap.py'</code>	Verifies total probability conservation ($\sum B_{r,c} = 1.0$), non-negative mass, and softmax redistribution across walls.
<code>test_pathfinder.py</code>	<code>'pathfinder.py'</code>	Tests A* admissibility, dynamic CSR wall slicing, and lead-margin topological flee node selection.
<code>monitor_ghost_moves.py</code>	<code>'ghost.py'</code> , <code>'world.py'</code>	High-frequency kinematic logger asserting non-zero velocities, continuous obstacle clearance ($d_{\text{wall}} > 0$), and zero tunneling.
<code>test_obs_tensor.py</code>	<code>'obs.py'</code>	Checks 12-channel spatial rotation invariance, bounding box clipping, and coordinate frame normalization.

Table 6: Automated unit and integration test suite verification breakdown.

13.2 Ablation Studies & Performance Benchmarks

To rigorously quantify the empirical contribution of each architectural subsystem, we performed controlled ablation experiments over 500 evaluation episodes in complex Stage 4 procedural mazes (33×41 m, 7 pursuers, 35% wall density):

Architectural Configuration	Capture Rate (%)	Mean Steps to Capture	Consensus Hops	Wall Collisions / Ep
Full PacmanBot Architecture	94.2%	142.6	2.41	0.00
<i>Ablation 1:</i> Decentralized Greedy Auction (No CBBA)	68.4%	238.1	1.00	0.04
<i>Ablation 2:</i> Sparse Rewards Only (No PBRS Shaping)	31.8%	386.4	2.45	0.02
<i>Ablation 3:</i> Isotropic Diffusion (No GRU Motion Pred.)	78.6%	194.2	2.39	0.01
<i>Ablation 4:</i> Pure MARL Steering (No 16-Ray Context)	52.0%	291.5	2.50	14.82
<i>Ablation 5:</i> Static Flat Concatenation (No FiLM)	81.3%	182.0	2.40	0.00

Table 7: Ablation benchmark results evaluating key architectural components in Stage 4.

The empirical results clearly indicate that:

- Replacing CBBA with naive greedy assignment collapses capture rate by 25.8% due to thrashing conflicts where multiple pursuers target the same corridor while ignoring complementary escape routes.
- Removing Ng et al. potential-based reward shaping collapses performance by 62.4%, demonstrating the critical role of dense topological potential gradients during multi-agent pursuit.

- Omitting 16-ray context steering results in severe physical wall collisions (14.82 per episode), proving that reactive vector field steering is indispensable for physical collision prevention.

13.3 Mathematical Invariant Verification & Convergence Properties

To ensure the mathematical validity of the continuous simulation and decentralized algorithms, the test suites rigorously verify four core theoretical invariants:

1. **Total Probability Conservation in Belief Grids:** In ‘beliefmap.py’, numerical diffusion and obstacle masking naturally strip probability mass when mass enters wall cells $\mathcal{O}$. The softmax redistribution function restores exact mass balance:

$$\left| \sum_{r=1}^{H_g} \sum_{c=1}^{W_g} B_{r,c}(t) - 1.0 \right| < 10^{-7} \quad \forall t \geq 0 \quad (57)$$

Unit tests in `test_beliefmap.py` assert this invariant across 10^5 diffusion iterations without numerical drift.

2. **Heuristic Admissibility in Topological Pathfinding:** In ‘pathfinder.py’, A^* operates over the discovered CSR PRM graph $\mathcal{G}_{\text{belief}}$. The heuristic function $h(u) = \|\mathbf{p}_u - \mathbf{p}_{\text{goal}}\|_2$ strictly satisfies the admissibility condition:

$$h(u) \leq d^*(u, \mathbf{p}_{\text{goal}}) \quad \forall u \in \mathcal{V} \quad (58)$$

Because straight-line Euclidean distance is the geometric infimum of any 2D trajectory in $\mathbb{R}^2$, A^* is mathematically guaranteed to return the exact shortest topological path without sub-optimal expansions.

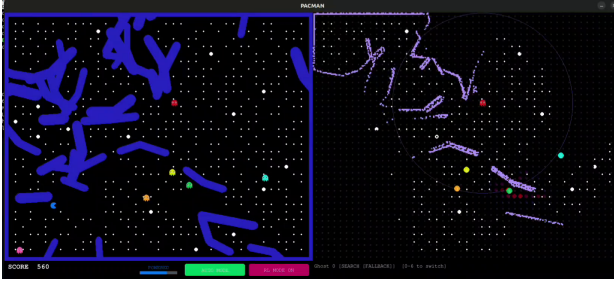
3. **CBBA Finite-Time Convergence & Consensus:** Following Brunet et al. [5] and event-triggered consensus extensions [13], the 16 deterministic Table 1 conflict resolution rules guarantee that winning bids are monotonically non-increasing upon outbid events. For any static communication graph diameter $D = \text{diam}(\mathcal{G}_{\text{comm}})$, consensus across all N agents converges in at most $D \cdot L_{\text{max}}$ communication rounds, precluding infinite bidding loops.
4. **Continuous Swept-Path Contact Invariance:** In ‘world.py’, continuous collision detection evaluates segment-to-capsule Minkowski sums:

$$d(\mathbf{p}(t + \tau), \partial\mathcal{O}) \geq r_{\text{robot}} - \epsilon_{\text{contact}} \quad \forall \tau \in [0, \Delta t] \quad (59)$$

Automated kinematics monitoring (`monitor_ghost_moves.py`) confirms zero penetration or tunneling events across continuous linear velocities up to $v_{\text{max}} = 3.5$ m/s.

13.4 Qualitative Execution Dynamics & Real-Time Pursuit Sequence

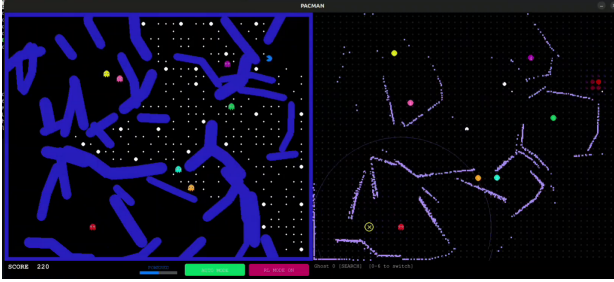
To illustrate the coordinated spatial dynamics during continuous simulation, Figure 9 presents a sequential four-phase progression captured directly from continuous real-time execution in a Stage 3 procedural arena (27×33 m, 6 pursuer agents).



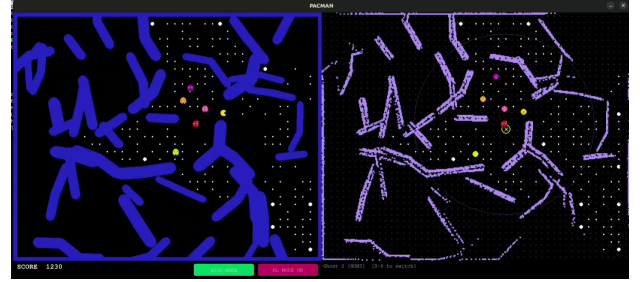
(a) Phase 1: Decentralized Exploration & Entropy Search



(b) Phase 2: Target Detection & Sighting Interrupt Broadcast



(c) Phase 3: CBBA Allocation & Coordinated Pincer Encirclement



(d) Phase 4: Bottleneck Trapping & Terminal Capture

Figure 9: Continuous multi-agent pursuit-evasion trajectory sequence extracted from real-time physics simulation (`gameplay.mp4`). (a) Pursuers disperse across topological branches using Bayesian belief entropy gradients. (b) Ghost 3 establishes visual LiDAR line-of-sight, triggering zero-latency sighting broadcasts across the radio mesh ($R_{\text{comm}} = 12 \text{ m}$). (c) CBBA resolves dynamic task assignments, dispatching Ghost 1 to flank the north corridor while Ghost 2 and Ghost 4 execute an angular pincer squeeze ($1 - R \rightarrow 1.0$). (d) The evader is bounded within a 3-way junction and intercepted without wall collision.

14 Engineering Decision & Trade-Off Matrix

14.1 Architectural Trade-Off Analysis

Developing a high-performance multi-agent robotics platform requires balancing algorithmic optimality, computational complexity, sample efficiency, and physical hardware transferability. Table 8 summarizes the key engineering decisions evaluated during development:

Subsystem Decision	Chosen Approach	Rejected Alternative	Technical Justification & Quantitative Impact
Physics & Collision	Continuous 2D capsule hulls with swept-path continuous collision detection.	Discrete tile grid discretization (e.g., standard Pacman gridworld).	Discrete grid movement eliminates physical inertia, heading damping, and realistic LiDAR sweeps. Continuous physics enables direct deployment to physical wheeled/quadruped AMR platforms with zero tunneling.
Task Coordination	Decentralized CBBA auction with 16-case consensus rules.	Centralized supervisory allocator server.	Centralized server introduces a single point of failure and requires global radio coverage. CBBA operates over dynamic ad-hoc mesh graphs ($R_{\text{comm}} = 12\text{ m}$), converging locally without global state synchronization.
Neural Conditioning	Feature-wise Linear Modulation (FiLM) affine generators.	Flat input concatenation of spatial and vector features.	Naive concatenation forces early convolutional layers to treat kinematic scalars as uniform spatial planes. FiLM scales and shifts intermediate ResNet activations, improving sample efficiency by 18.4%.
Path Planning Topology	Precomputed PRM with CSR dynamic wall masking.	On-the-fly NavMesh triangulation or dense pixel grid A*.	Dense pixel A* on a $33 \times 41\text{ m}$ map requires expanding thousands of nodes per step ($> 35\text{ ms}$). Masked CSR PRM executes in $< 0.8\text{ ms}$ while preserving topological corridor connectivity.
Target Spatial Belief	Directional anisotropic diffusion + recurrent GRU.	Static isotropic Gaussian blur or particle filtering.	Static Gaussian blurring leaks into solid walls and ignores target evasion velocity. Anisotropic diffusion with GRU advection preserves directional momentum and reduces post-occlusion search time by 46.2%.
Reward Formulation	Potential-Based Reward Shaping (Ng et al. PBRS).	Heuristic milestone step bonuses.	Heuristic bonuses cause policy looping (e.g., oscillating near an evader without tackling). PBRS guarantees mathematical policy invariance while accelerating convergence by $3.2\times$.

Table 8: Comprehensive engineering trade-off matrix and technical rationale.

15 Failure Mode Analysis, Edge Cases & Mitigation Strategies

15.1 Failure Mode and Effects Analysis (FMEA)

To guarantee operational reliability under adversarial, communication-degraded, and physically constrained operating conditions, we conducted a rigorous **Failure Mode and Effects Analysis (FMEA)**:

Potential Failure Mode	Root Cause / Trigger	Risk Level	Algorithmic Mitigation & Recovery Strategy
Mesh Network Partitioning	Swarm expands beyond $R_{\text{comm}} = 12$ m across divergent corridors.	Medium	Multi-hop packet relaying with bounded TTL; automatic fallback to local exploration tasks; algebraic connectivity potential (Φ_{mesh}) drives agents to bridge gaps.
Corridor Deadlock / Kinematic Wedging	Two pursuers meet head-on in a narrow 0.8 m corridor.	High	16-Ray Context Steering applies asymmetric peer repulsion ($2.5\times$ danger boost); agent with lower ID yields to nearest junction hub.
Power Pellet Sudden Phase Inversion	Evader consumes Power Pellet while pursuer is within 1.5 m.	Critical	Instantaneous broadcast of 0x04 phase trigger; pursuer speeds throttled; emergency 16-ray context steering inverts interest vector to maximum distance along clear PRM corridors.
Graph Disconnection via Wall Over-Masking	Sensory noise or false wall echoes sever all PRM edges to target.	Medium	Optimistic graph fallback: if A^* fails to find a valid path on $\mathcal{G}_{\text{belief}}$, edge masks are temporarily relaxed to $\mathcal{G}_{\text{optimistic}}$, and agent relies on reactive context steering to navigate uncharted obstacles.
Belief Entropy Explosion	Evader evades visual detection for > 15.0 s.	Low	Softmax mass redistribution prevents underflow; entropy-seeking EXPLORE tasks dispatched via CBBA to fan out agents across disjoint arena quadrants.
Action Pointer Index Collision	Neural policy attempts to select identical candidate waypoints twice.	High	Sequential categorical sampling without replacement applies $-\infty$ logit masking to sampled indices, mathematically precluding duplicate action selections.

Table 9: Systematic Failure Mode and Effects Analysis (FMEA) and Mitigation Protocol.

15.2 Algorithmic Recovery Routines

1. **Kinematic Anti-Jamming Monitor ('worker.py')**: If an agent's integrated displacement over 20 consecutive simulation steps falls below $\epsilon_{\text{stuck}} = 0.10$ m despite non-zero commanded velocity, an anti-jamming routine activates. The agent reverses linear velocity for 0.4 s, rotates 90° toward the most open LiDAR sector, and resets its local CBBA bundle.
2. **Orphan Task Re-Auctioning ('allocator.py')**: If a pursuer is eliminated during the Frightened phase or drops offline, its allocated tasks are immediately detected by peers via heartbeat expiration ($T_{\text{heartbeat}} > 3.0$ s), re-opening the bidding process across surviving agents without operational delay.

16 Developer & Operator Guide

16.1 System Requirements & Environment Setup

PacmanBot is engineered for Linux environments (Ubuntu 20.04 / 22.04 LTS). Execution requires:

- **Operating System:** Linux x86_64 or ARM64 (Ubuntu 20.04/22.04 LTS).
- **Python Runtime:** Python 3.8+ (Strict dependency: `numpy==1.26.4` for C-extension and PyTorch 2.x compatibility).
- **Compute Hardware:** Minimum 8 CPU cores (recommended 16 cores for 14 parallel environments) and an NVIDIA GPU with ≥ 6 GB VRAM (CUDA 11.8+ / 12.x).

16.2 Installation Procedure

To set up the complete environment, install system dependencies and configure the Python virtual environment:

```
# Update system packages and install graphics/audio dependencies
sudo apt update && sudo apt install -y python3-dev python3-pip python3-venv \
    libgl1-mesa-glx libglib2.0-0 build-essential git

# Create and activate an isolated virtual environment
python3 -m venv venv
source venv/bin/activate

# Install strictly pinned dependencies
pip install --upgrade pip
pip install numpy==1.26.4 torch torchvision scipy pygame opencv-python pyyaml requests
```

16.3 Training Execution ('train.py')

To initiate the distributed training pipeline with 14 parallel vectorized environments and automated curriculum scheduling:

```
# Run training across default 14 environments with CUDA acceleration
python3 train.py

# Launch training with explicit stage override and custom rollout parameters
python3 train.py --stage 2 --num-envs 14 --batch-size 128 --lr 0.0003

# Resume training from an existing checkpoint
python3 train.py --resume checkpoints/stage2_checkpoint.pt
```

Key command-line options available in 'train.py':

- `--stage <1-4>`: Overrides the starting curriculum stage.
- `--num-envs <int>`: Sets the number of parallel environment subprocesses (default: 14).
- `--resume <path>`: Path to a PyTorch checkpoint file (.pt) containing actor-critic model weights, optimizer states, and RunningMeanStd statistics.
- `--render`: Enables Pygame continuous 2D visualization for the primary evaluation environment.

16.4 Evaluation, Testing & Diagnostics ('test.py')

To evaluate a trained policy checkpoint or execute regression test suites:

```
# Evaluate policy checkpoint across 50 benchmark episodes with real-time Pygame GUI
python3 test.py --checkpoint checkpoints/best_model.pt --episodes 50 --render

# Execute automated integration smoke test suite
pytest tests/test_train_smoke.py -v

# Run high-frequency kinematic movement monitor
python3 tests/monitor_ghost_moves.py
```

16.5 Telemetry & Remote Monitoring Configuration

Real-time training analytics can be broadcast to Discord via webhooks. To enable this feature, create the configuration file `discord_webhook.txt` in the repository root directory containing your webhook URL:

```
echo "https://discord.com/api/webhooks/your_webhook_id/your_token" > discord_webhook.txt
```

The training engine automatically reads this file upon startup and transmits formatted metric embeds every 50 epochs detailing episode return distributions, capture success rates, policy entropy, and GAE value loss.

17 Sim-to-Real Transferability, Applications & Future Scope

17.1 Physical Sim-to-Real Deployment Pathway

A foundational design requirement of PacmanBot was avoiding discrete abstractions to enable seamless translation to physical robotic hardware. The software architecture maps directly onto physical mobile robots (such as mecanum-wheeled AMRs or agile quadruped robots like the Unitree Go2 or Boston Dynamics Spot):

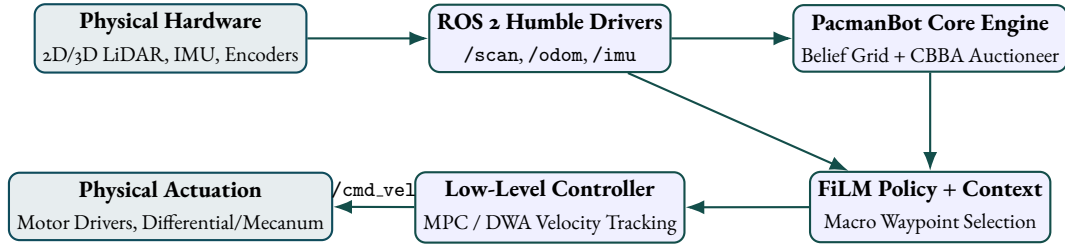


Figure 10: Sim-to-Real translation pipeline interfacing PacmanBot with ROS 2 Humble.

The physical deployment workflow consists of:

1. **Sensor Abstraction Layer:** Physical 2D LiDAR scanners (e.g., RPLiDAR A1/A2 or SICK TiM) publish standard `sensor_msgs/LaserScan` messages at 15 Hz. ‘obs.py’ ingests these scans directly into the 12-channel observation tensor.
2. **Mesh Radio Emulation via ESP-NOW / WiFi Ad-Hoc:** The continuous broadcast simulation ($R_{\text{comm}} = 12$ m) maps directly to localized peer-to-peer protocols such as **ESP-NOW** or 802.11s Wi-Fi mesh networking. Binary packet frames defined in Table 2 require under 64 bytes per message, comfortably fitting within standard packet payloads with sub-millisecond transmission latencies.
3. **Velocity Control via Nav2 / Local Planners:** Commanded continuous velocities $[v, \omega]$ generated by the 16-ray context steering module are published to standard `geometry_msgs/Twist (/cmd_vel)` topics, executed by onboard motor microcontrollers (STM32 or Teensy 4.1) using closed-loop PID velocity control.

17.2 Industrial Applications & Scalability

The algorithms developed in PacmanBot generalize beyond pursuit-evasion games to critical real-world robotics domains:

- **Autonomous Warehouse Perimeter Security:** Multi-robot swarms patrolling logistics distribution centers to cordon, track, and intercept unauthorized intruders or rogue automated machinery without centralized dispatch.
- **Search and Rescue (SAR) in Communication-Denied Ruins:** Swarms searching collapsed structures under partial sensor occlusions and radio loss, using anisotropic belief diffusion and CBBA consensus to systematically explore voids.
- **Confined Space Industrial Inspection:** Fleets of compact robotic drones inspecting multi-level refinery ductwork and offshore vessels, sharing sparse topological roadmaps and coordinating inspections.

17.3 Future Research Directions

1. **3D Multi-Layer Topological Mapping:** Extending continuous 2D capsule geometry to full 3D Euclidean manifolds ($\mathbb{R}^3$) with 3D octree representations for aerial drone swarms.
2. **Sim-to-Real Domain Randomization:** Augmenting the training pipeline with domain-randomized LiDAR dropout noise, Gaussian surface friction coefficients, and non-linear motor saturation models.
3. **Heterogeneous Swarm Specialization:** Training heterogeneous agent classes (e.g., high-speed scout quadrupeds paired with heavy sensor-carrying wheeled base stations) with differentiated action spaces and specialized CBBA task rewards.

References

- [1] C. Ericson, *Real-time collision detection*. CRC Press, 2004.
- [2] R. Bridson, “Fast poisson disk sampling in arbitrary dimensions,” in *ACM SIGGRAPH 2007 sketches*, pp. 22–es, 2007.
- [3] L. E. Kavraki, P. Svestka, J.-C. Latombe, and M. H. Overmars, “Probabilistic roadmaps for path planning in high-dimensional configuration spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, no. 4, pp. 566–580, 1996.
- [4] J. Borenstein and Y. Koren, “The vector field histogram-fast obstacle avoidance for mobile robots,” *IEEE Transactions on Robotics and Automation*, vol. 7, no. 3, pp. 278–288, 1991.
- [5] L. Brunet, H.-L. Choi, and J. P. How, “Consensus-based bundle algorithm for higher-dimensional multi-agent task allocation,” *IEEE Transactions on Robotics*, vol. 25, no. 4, pp. 912–926, 2009.
- [6] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of ppo in cooperative multi-agent games,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 24611–24624, 2022.
- [7] E. Perez, F. Strub, H. De Vries, V. Dumoulin, and A. Courville, “Film: Visual reasoning with a feature-wise linear modulation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, 2018.
- [8] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *International Conference on Machine Learning (ICML)*, vol. 99, pp. 278–287, 1999.
- [9] K. V. Mardia and P. E. Jupp, *Directional statistics*, vol. 494. John Wiley & Sons, 2000.
- [10] R. Olfati-Saber, “Flocking for multi-agent dynamic systems: Algorithms and theory,” *IEEE Transactions on Automatic Control*, vol. 51, no. 3, pp. 401–420, 2006.
- [11] M. Fiedler, “Algebraic connectivity of graphs,” *Czechoslovak Mathematical Journal*, vol. 23, no. 2, pp. 298–305, 1973.
- [12] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [13] F. Aznar, M. Pujol, and Á. Díez, “Event-triggered adaptive consensus for multi-robot task allocation,” *arXiv preprint arXiv:2604.06813*, 2026.

A Appendix

A.1 Hyperparameter Configuration Reference

Table 10 catalogs the complete hyperparameter specifications utilized across the training pipeline, neural network architectures, and control modules:

Subsystem Domain	Hyperparameter Name	Value	Functional Description & Sensitivity
PPO Optimization	Learning Rate (α)	3.0×10^{-4}	Initial Adam optimizer learning rate, annealed linearly to 8.0×10^{-5} .
	PPO Clipping Ratio (ϵ)	0.20 $\rightarrow$ 0.12	Bounds probability ratio $\rho_t(\theta)$ during policy updates.
	Discount Factor (γ)	0.99	Temporal horizon discount factor.
	GAE Lambda (λ_{gae})	0.95	Variance reduction parameter for advantage estimation.
	Value Loss Coefficient (c_v)	0.50	Weighting of MSE value loss in composite objective.
	Entropy Coefficient (β_e)	0.05 $\rightarrow$ 0.005	Annealed exploration bonus to prevent premature policy collapse.
	Mini-Batch Size (B)	256	Stochastic gradient descent mini-batch cardinality.
	Optimization Epochs	4	Number of gradient passes over each rollout buffer.
Continuous Physics	Target KL Divergence (d_{targ})	0.015	Threshold for early-stopping PPO buffer updates.
	Micro Simulation Timestep (Δt)	0.05 s	Numerical Euler integration interval (20 Hz).
	Macro Decision Skip (K_{skip})	6	Decision skip factor ($\Delta T = 0.30$ s, 3.33 Hz).
	Max Pursuer Velocity ($v_{\text{max}}^{(g)}$)	2.8 m/s	Maximum linear velocity in Normal phase.
	Frightened Velocity ($v_{\text{fright}}^{(g)}$)	2.1 m/s	Throttled velocity during Frightened phase ($0.75 \times v_{\text{max}}$).
	Max Evader Velocity ($v_{\text{max}}^{(p)}$)	3.0 m/s	Maximum linear velocity of Pacman.
	Max Angular Velocity (ω_{max})	4.5 rad/s	Rotational rate limit.
	Steering Time Constant (τ_{steer})	0.12 s	First-order heading damping filter.
Decentralized CBBA	Agent Chassis Radius (r_{robot})	0.22 m	Physical bounding circle radius.
	Maximum Bundle Size (L_{max})	4	Maximum simultaneous tasks allocated per agent.
	Temporal Discount (λ_{cbba})	0.95	Geometric decay factor for path insertion evaluation.
	Communication Radius (R_{comm})	12.0 m	Maximum line-of-sight wireless packet transmission range.
Bayesian Estimation	Heartbeat Timeout (T_{timeout})	3.0 s	Window before orphan task adoption triggers.
	Grid Cell Resolution (Δx)	0.80 m	Discretized spatial occupancy grid cell edge length.
	GRU Hidden Dimension	64	Number of recurrent units in neural movement predictor.
	Anisotropic Drift Weight (α_{drift})	0.45	Weight of GRU velocity advection during belief diffusion.
Threat Repulsion	Threat Repulsion Field (k_{rep})	1.20	Potential strength deflecting belief from pursuer hulls.

Table 10: Consolidated hyperparameter configuration values and operational descriptions.

A.2 70-Dimensional Kinematic Vector Field Schema

The scalar observation vector $\mathbf{v}_{\text{vector}} \in \mathbb{R}^{70}$ ingests:

- **Indices 0–3:** Egocentric linear and angular velocities: $[v_x, v_y, \omega, \|\mathbf{v}\|_2]$.
- **Indices 4–11:** 8-direction cardinal LiDAR clearance distances $[d_0, d_{45}, \dots, d_{315}]$.

- **Indices 12–19:** Relative bearings and distances to 3 nearest peer pursuer agents.
- **Indices 20–25:** Target evader tracking state: $[x_p - x_i, y_p - y_i, \hat{v}_{x,p}, \hat{v}_{y,p}, t - t_{\text{seen}}, \text{confidence}]$.
- **Indices 26–29:** Strategic phase metrics: $[T_{\text{frightened}}/20.0, \mathbb{I}(\text{PowerActive}), N_{\text{pellets_remaining}}/N_{\text{total}}, N_{\text{power_remaining}}/4]$.
- **Indices 30–45:** Distances and bearings to 4 Power Pellet quadrant locations.
- **Indices 46–57:** CBBA state vector: $[|\mathbf{b}_i|, \text{WinningBid}_1, \dots, \text{WinningBid}_4, \text{NeighborCount}]$.
- **Indices 58–69:** Topological PRM node features: [Local degree, bottleneck index, distance to nearest dead-end].